



[DOCUMENT TITLE]

[Document subtitle]



Contents

Lab 5 Honeypots:	1
Part 1 Password Guessing With Hydra:	1
Part 2 Logs:	4
Part 3 Run Hydra Against Cowrie:	5
Part 4 SSH Into Cowrie:	6
Part 5 Cowrie logs:	8
Task 6 Download a file inside cowrie:	9
Task 7 Exit from cowrie ssh session:	10
Task 8 SSH out of cowrie:	10
Task 9 Cowrie Files:	11
Lab 6 NetFlow:	13
Lab 8 PCAP: Drive by Download	20
Part 1 Drive by Download Without FAST FLUX:	20
Part 2 Drive by Download with FAST FLUX:	26
Part 3 Tshark:	38

Lab 5 Honeypots:

Part 1 Password Guessing with Hydra:

First, we create the 2 files users and passwords:

```
cyuser@kali: ~  
File Actions Edit View Help  
GNU nano 4.9.2 users  
#sfadmin  
root  
adhd  
cyuser  
student  
[ Read 5 lines ]  
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify  
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell
```

```
cyuser@kali: ~  
File Actions Edit View Help  
GNU nano 4.9.2 passwords  
root  
123456  
SamplePass1  
[ Read 3 lines ]  
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify  
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell
```

Next we must get our ip of our host computer so we can to run hydra to perform our password guessing attack

```

cyuser@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group d
efault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
UP group default qlen 1000
    link/ether 2e:91:98:5e:9f:c9 brd ff:ff:ff:ff:ff:ff
    inet 10.10.1.5/24 brd 10.10.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::5/64 scope link
        valid_lft forever preferred_lft forever
    inet6 fe80::2c91:98ff:fe5e:9fc9/64 scope link
        valid_lft forever preferred_lft forever
cyuser@kali:~$

```

Using “hydra -L users -P passwords 10.10.1.5 ssh -V” we are able to perform the attack on the host to view the passwords as seen in this screenshot below.

```

cyuser@kali:~$ hydra -L users -P passwords 10.10.1.5 ssh -V
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or se
cret service organizations, or for illegal purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2023-10-25 0
2:43:33
[WARNING] Many SSH configurations limit the number of parallel tasks, it is
recommended to reduce the tasks: use -t 4
[DATA] max 15 tasks per 1 server, overall 15 tasks, 15 login tries (l:5/p:3
), ~1 try per task
[DATA] attacking ssh://10.10.1.5:22/
[ATTEMPT] target 10.10.1.5 - login "msfadmin" - pass "root" - 1 of 15 [chil
d 0] (0/0)
[ATTEMPT] target 10.10.1.5 - login "msfadmin" - pass "123456" - 2 of 15 [ch
ild 1] (0/0)
[ATTEMPT] target 10.10.1.5 - login "msfadmin" - pass "SamplePass1" - 3 of 1
5 [child 2] (0/0)
[ATTEMPT] target 10.10.1.5 - login "root" - pass "root" - 4 of 15 [child 3]
(0/0)
[ATTEMPT] target 10.10.1.5 - login "root" - pass "123456" - 5 of 15 [child
4] (0/0)
[ATTEMPT] target 10.10.1.5 - login "root" - pass "SamplePass1" - 6 of 15 [c
hild 5] (0/0)
[ATTEMPT] target 10.10.1.5 - login "adhd" - pass "root" - 7 of 15 [child 6]
(0/0)
[ATTEMPT] target 10.10.1.5 - login "adhd" - pass "123456" - 8 of 15 [child
7] (0/0)
[ATTEMPT] target 10.10.1.5 - login "adhd" - pass "SamplePass1" - 9 of 15 [c
hild 8] (0/0)
[ATTEMPT] target 10.10.1.5 - login "cyuser" - pass "root" - 10 of 15 [child
9] (0/0)
[ATTEMPT] target 10.10.1.5 - login "cyuser" - pass "123456" - 11 of 15 [chi
ld 10] (0/0)
[ATTEMPT] target 10.10.1.5 - login "cyuser" - pass "SamplePass1" - 12 of 15
[child 11] (0/0)
[ATTEMPT] target 10.10.1.5 - login "student" - pass "root" - 13 of 15 [chil
d 12] (0/0)
[ATTEMPT] target 10.10.1.5 - login "student" - pass "123456" - 14 of 15 [ch
ild 13] (0/0)
[ATTEMPT] target 10.10.1.5 - login "student" - pass "SamplePass1" - 15 of 1
5 [child 14] (0/0)
[REDO-ATTEMPT] target 10.10.1.5 - login "msfadmin" - pass "123456" - 16 of
17 [child 1] (1/2)
[REDO-ATTEMPT] target 10.10.1.5 - login "root" - pass "123456" - 17 of 17 [
child 4] (2/2)
[22][ssh] host: 10.10.1.5 login: cyuser password: SamplePass1
[REDO-ATTEMPT] target 10.10.1.5 - login "root" - pass "123456" - 18 of 18 [
child 4] (3/3)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2023-10-25 0
2:19:10

```


Part 2 Logs:

- What IP address was the attack launched from?

- 10.10.1.4 (our attacker vm)

- How many valid usernames were attacked?

- There was 1 valid user and that was cyuser

```
Oct 25 02:44:30 kali sshd[1418]: pam_unix(sshd:session): session opened for user cyuser by (uid=0)
```

- How many invalid usernames were attacked?

- There were 3 invalid users and they were msfadmin, adhd and student

```
Oct 25 02:44:30 kali sshd[1411]: Invalid user adhd from 10.10.1.4 port 35800
Oct 25 02:44:30 kali sshd[1408]: Invalid user msfadmin from 10.10.1.4 port 35794
Oct 25 02:44:30 kali sshd[1419]: Invalid user student from 10.10.1.4 port 35816
```

- How many unsuccessful attacks do you see in the log?

- Altogether there was 25 unsuccessful attacks

- Here are some unsuccessful password attempts

```
Oct 25 02:44:32 kali sshd[1411]: Failed password for invalid user adhd from 10.10.1.4 port 35800 ssh2
Oct 25 02:44:32 kali sshd[1419]: Failed password for invalid user student from 10.10.1.4 port 35816 ssh2
Oct 25 02:44:32 kali sshd[1413]: Failed password for root from 10.10.1.4 port 35804 ssh2
Oct 25 02:44:32 kali sshd[1417]: Failed password for cyuser from 10.10.1.4 port 35810 ssh2
Oct 25 02:44:32 kali sshd[1408]: Failed password for invalid user msfadmin from 10.10.1.4 port 35794 ssh2
Oct 25 02:44:32 kali sshd[1412]: Failed password for invalid user adhd from 10.10.1.4 port 35802 ssh2
Oct 25 02:44:32 kali sshd[1407]: Failed password for invalid user msfadmin from 10.10.1.4 port 35792 ssh2
Oct 25 02:44:32 kali sshd[1409]: Failed password for root from 10.10.1.4 port 35796 ssh2
Oct 25 02:44:32 kali sshd[1428]: Failed password for invalid user student from 10.10.1.4 port 35826 ssh2
Oct 25 02:44:32 kali sshd[1414]: Failed password for cyuser from 10.10.1.4 port 35806 ssh2
Oct 25 02:44:32 kali sshd[1416]: Failed password for root from 10.10.1.4 port 35808 ssh2
Oct 25 02:44:32 kali sshd[1411]: Connection closed by invalid user adhd 10.10.1.4 port 35800 [preauth]
Oct 25 02:44:32 kali sshd[1425]: Failed password for invalid user student from 10.10.1.4 port 35824 ssh2
Oct 25 02:44:32 kali sshd[1408]: Connection closed by invalid user msfadmin 10.10.1.4 port 35794 [preauth]
Oct 25 02:44:32 kali sshd[1407]: Connection closed by invalid user msfadmin 10.10.1.4 port 35792 [preauth]
Oct 25 02:44:32 kali sshd[1413]: Connection closed by authenticating user root 10.10.1.4 port 35804 [preauth]
Oct 25 02:44:32 kali sshd[1409]: Connection closed by authenticating user root 10.10.1.4 port 35796 [preauth]
Oct 25 02:44:32 kali sshd[1406]: Failed password for invalid user msfadmin from 10.10.1.4 port 35790 ssh2
Oct 25 02:44:32 kali sshd[1410]: Failed password for invalid user adhd from 10.10.1.4 port 35798 ssh2
Oct 25 02:44:32 kali sshd[1416]: Connection closed by authenticating user root 10.10.1.4 port 35808 [preauth]
```

- Some authentication failures

```

Oct 25 02:44:30 kali sshd[1411]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4
Oct 25 02:44:30 kali sshd[1419]: pam_unix(sshd:auth): check pass; user unknown
Oct 25 02:44:30 kali sshd[1419]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4
Oct 25 02:44:30 kali sshd[1413]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4 user=root
Oct 25 02:44:30 kali sshd[1417]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4 user=cyuser
Oct 25 02:44:30 kali sshd[1412]: pam_unix(sshd:auth): check pass; user unknown
Oct 25 02:44:30 kali sshd[1408]: pam_unix(sshd:auth): check pass; user unknown
Oct 25 02:44:30 kali sshd[1408]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4
Oct 25 02:44:30 kali sshd[1407]: pam_unix(sshd:auth): check pass; user unknown
Oct 25 02:44:30 kali sshd[1407]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4
Oct 25 02:44:30 kali sshd[1412]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4
Oct 25 02:44:30 kali sshd[1409]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4 user=root
Oct 25 02:44:30 kali sshd[1428]: Invalid user student from 10.10.1.4 port 35826
Oct 25 02:44:30 kali sshd[1428]: pam_unix(sshd:auth): check pass; user unknown
Oct 25 02:44:30 kali sshd[1428]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4
Oct 25 02:44:30 kali sshd[1414]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4 user=cyuser
Oct 25 02:44:30 kali sshd[1416]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=10.10.1.4 user=root
Oct 25 02:44:30 kali sshd[1418]: Accepted password for cyuser from 10.10.1.4 port 35814 ssh2

```

- How many successful attacks?
 - Only 1 successful attack

```

Oct 25 02:44:30 kali sshd[1418]: Accepted password for cyuser from 10.10.1.4 port 35814 ssh2
Oct 25 02:44:30 kali sshd[1418]: pam_unix(sshd:session): session opened for user cyuser by (uid=0)

```

Part 3 Run Hydra Against Cowrie:

For this part I made sure to add `listen_port = 2222` into the `cowrie.cfg` file so it could listen to the port 2222 and we can attempt to run our hydra attack through this against cowrie.

After this part we must start the cowrie by first entering the right directory so first I use the command `cd /opt/cowrie` and then the command `bin/cowrie start` to start it and our new port will also come into effect as we will see in these screenshots


```
adhd@adhd:~$ ss -ttn
tcp        0      0 0.0.0.0:44443        0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:1723        0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:16993       0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:5060        0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:5061        0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:5060        0.0.0.0:*            LISTEN
tcp        0      0 127.0.0.1:3306       0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:5900        0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:2222        0.0.0.0:*            LISTEN
tcp        0      0 0.0.0.0:110         0.0.0.0:*            LISTEN
tcp6       0      0 :::80               :::*                  LISTEN
tcp6       0      0 :::11631            :::*                  LISTEN
```

```
cyuser@kali:~$ hydra -L users -P passwords 10.10.1.6 ssh -s 2222 -V
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2023-10-25 07:16:15
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 15 tasks per 1 server, overall 15 tasks, 15 login tries (l:5/p:3), ~1 try per task
[DATA] attacking ssh://10.10.1.6:2222/
[ATTEMPT] target 10.10.1.6 - login "msfadmin" - pass "root" - 1 of 15 [child 0] (0/0)
[ATTEMPT] target 10.10.1.6 - login "msfadmin" - pass "123456" - 2 of 15 [child 1] (0/0)
[ATTEMPT] target 10.10.1.6 - login "msfadmin" - pass "SamplePass1" - 3 of 15 [child 2] (0/0)
[ATTEMPT] target 10.10.1.6 - login "root" - pass "root" - 4 of 15 [child 3] (0/0)
[ATTEMPT] target 10.10.1.6 - login "root" - pass "123456" - 5 of 15 [child 4] (0/0)
[ATTEMPT] target 10.10.1.6 - login "root" - pass "SamplePass1" - 6 of 15 [child 5] (0/0)
[ATTEMPT] target 10.10.1.6 - login "adhd" - pass "root" - 7 of 15 [child 6] (0/0)
[ATTEMPT] target 10.10.1.6 - login "adhd" - pass "123456" - 8 of 15 [child 7] (0/0)
[ATTEMPT] target 10.10.1.6 - login "adhd" - pass "SamplePass1" - 9 of 15 [child 8] (0/0)
[ATTEMPT] target 10.10.1.6 - login "cyuser" - pass "root" - 10 of 15 [child 9] (0/0)
[ATTEMPT] target 10.10.1.6 - login "cyuser" - pass "123456" - 11 of 15 [child 10] (0/0)
[ATTEMPT] target 10.10.1.6 - login "cyuser" - pass "SamplePass1" - 12 of 15 [child 11] (0/0)
[ATTEMPT] target 10.10.1.6 - login "student" - pass "root" - 13 of 15 [child 12] (0/0)
[ATTEMPT] target 10.10.1.6 - login "student" - pass "123456" - 14 of 15 [child 13] (0/0)
[ATTEMPT] target 10.10.1.6 - login "student" - pass "SamplePass1" - 15 of 15 [child 14] (0/0)
[2222][ssh] host: 10.10.1.6 login: root password: SamplePass1
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2023-10-25 07:16:17
```

Part 4 SSH Into Cowrie:

```
cyuser@kali:~$ ssh root@10.10.1.6 -p 2222
The authenticity of host '[10.10.1.6]:2222 ([10.10.1.6]:2222)' can't be established.
RSA key fingerprint is SHA256:S1dZq3T492ut2IKGzANR4STQx0fnYYRp8SB5tbsU2kE.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.10.1.6]:2222' (RSA) to the list of known hosts.
root@10.10.1.6's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# ls /opt
root@svr04:~# ls /var
backups  cache  lib      local   lock    log     mail    opt     run     spool   tmp
```

```
adhd@adhd:~$ ls /opt
artillery          decloak            java-web-attack    rubberglue
BearTrap           denyhosts          metasploit-framework set
beef               gcat              opencanary         spidertrap
canarytokens-docker ghostwriting        PHP-HTTP-Tarpit    tcprooter
containerd         honeybadger        portspooft         webbugserver
cowrie             honeyports         psad               weblabyrinth
creepy            jar-combiner       recon-ng           wordpot

adhd@adhd:~$ ls /var
artillery  cache  lib  lock  mail  opt  snap  tmp
backups    crash  local log  metrics run  spool  www
adhd@adhd:~$
```

So in the first screenshot it shows that we ssh'd our way into adhd with the information we gathered off the last task and the differences we see here between the directory listings are due to Cowrie being a honeypot, which is designed to present a simulated environment to attackers while logging their actions. The real directories on the ADHD VM contain different data because they are from the actual system.

```
2023-10-25T05:20:56.711471Z [HoneyPotSSHTransport,49,10.10.1.4] starting service b'ssh-userauth'
2023-10-25T05:20:56.715046Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] b'root' tr
ying auth b'none'
2023-10-25T05:21:16.474699Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] b'root' tr
ying auth b'password'
2023-10-25T05:21:16.476378Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] Could not
read etc/userdb.txt, default database activated
2023-10-25T05:21:16.477621Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] login atte
mpt [b'root'/b'SamplePass1'] succeeded
2023-10-25T05:21:16.478594Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] Initialize
d emulated server as architecture: linux-x64-lsb
2023-10-25T05:21:16.479276Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] b'root' au
thenticated with b'password'
2023-10-25T05:21:16.479573Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,49,10.10.1.4] starting s
ervice b'ssh-connection'
2023-10-25T05:21:16.481565Z [SSHSservice b'ssh-connection' on HoneyPotSSHTransport,49,10.10.1.4] got chan
nel b'session' request
2023-10-25T05:21:16.482044Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] channel open
2023-10-25T05:21:16.482271Z [SSHSservice b'ssh-connection' on HoneyPotSSHTransport,49,10.10.1.4] got glob
al b'no-more-sessions@openssh.com' request
2023-10-25T05:21:16.598745Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] pty request: b'xterm-256color' (41, 123, 0, 0)
2023-10-25T05:21:16.598977Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] Terminal Size: 123 41
2023-10-25T05:21:16.599868Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] request_env: b'LANG'=b'en_US.utf8'
2023-10-25T05:21:16.600513Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] getting shell
2023-10-25T05:21:36.902640Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] CMD: ls /opt
2023-10-25T05:21:36.905293Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] Command found: ls /opt
2023-10-25T05:21:45.336274Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] CMD: ls /var
2023-10-25T05:21:45.338453Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,49,10.10.1.4] Command found: ls /var
```

Here we see entries of our username entered ip address we also see here the command “ls /opt” and “ls /var” showing we found our log on cowrie.

Part 5 Cowrie logs:

```
2023-10-25T05:44:59.671875Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] Command found: id
2023-10-25T05:45:06.210241Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] CMD: uname -a
2023-10-25T05:45:06.212748Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] Command found: uname -a
2023-10-25T05:45:16.805705Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] CMD: cat /proc/cpuinfo
2023-10-25T05:45:16.808089Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] Command found: cat /proc/cpuinfo
2023-10-25T05:45:24.582187Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] CMD: cat /proc/meminfo
2023-10-25T05:45:24.584594Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] Command found: cat /proc/meminfo
2023-10-25T05:45:35.398629Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] CMD: cat /etc/passwd
2023-10-25T05:45:35.400681Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] Command found: cat /etc/passwd
2023-10-25T05:45:42.530186Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] CMD: cat /etc/shadow
2023-10-25T05:45:42.532106Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,50,10.10.1.4] Command found: cat /etc/shadow
```

Here is the monitoring of the cowrie.log on the adhd vm and from the attacker vm in the ssh of the adhd we entered the following commands, id, uname -a, cat /proc/cpuinfo, cat /proc/meminfo, cat /etc/passwd, cat /etc/shadow and in the screenshot above we see the commands entered on the attacker have been monitored in the cowrie.log keeping track of the attackers activity.

```
root@svr04:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
richard:x:1000:1000:Richard Texas,,,:/home/richard:/bin/bash
root@svr04:~# cat /etc/shadow
root:$6$aOmWdpJ$kyP0ik9rR0kSLyABIYNXgg/UqLWX3c1eIaov0LWphShTGXmuUAMq6iu9DrcQqLVUw3Pirizns4u27w3Ugvb6.:15800:0:99999:7:::
daemon:!:15800:0:99999:7:::
bin:!:15800:0:99999:7:::
sys:!:15800:0:99999:7:::
sync:!:15800:0:99999:7:::
games:!:15800:0:99999:7:::
man:!:15800:0:99999:7:::
lp:!:15800:0:99999:7:::
mail:!:15800:0:99999:7:::
news:!:15800:0:99999:7:::
uucp:!:15800:0:99999:7:::
proxy:!:15800:0:99999:7:::
www-data:!:15800:0:99999:7:::
backup:!:15800:0:99999:7:::
list:!:15800:0:99999:7:::
irc:!:15800:0:99999:7:::
gnats:!:15800:0:99999:7:::
```

Task 6 Download a file inside cowrie:

```
cyuser@kali:~$ sudo service apache2 start
[sudo] password for cyuser:
cyuser@kali:~$ md5sum /var/www/html/index.html
e2620d4a5a0f8d80dd4b16de59af981f /var/www/html/index.html
```

```
cyuser@kali:~$ ssh root@10.10.1.6 -p 2222
root@10.10.1.6's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# wget http://10.10.1.5/index.html
--2023-10-25 06:06:40-- http://10.10.1.5/index.html
Connecting to 10.10.1.5:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10701 (10K) [text/html]
Saving to: '/root/index.html'

100%[=====>] 10,701      888K/s/s  eta 0s

2023-10-25 06:06:40 (888 KB/s) - '/root/index.html' saved [10701/10701]

root@svr04:~# pwd
/root
root@svr04:~# ls
index.html
root@svr04:~#
```

```
adhd@adhd:~$ cd /opt/cowrie/var/lib/cowrie/downloads
adhd@adhd:/opt/cowrie/var/lib/cowrie/downloads$ ls -l
total 12
-rw-r--r-- 1 adhd adhd 10701 Oct 25 06:06 f14e8167f12be74330c1b881b5aa3df95f5bd66d26f42cc03b87a7c38946c571
adhd@adhd:/opt/cowrie/var/lib/cowrie/downloads$
```

Cowrie successfully saved the downloaded file in its downloads directory, demonstrating that Cowrie logs and captures all downloaded files for later analysis.

```
adhd@adhd:~$ cd /opt/cowrie/var/lib/cowrie/downloads
adhd@adhd:/opt/cowrie/var/lib/cowrie/downloads$ ls
f14e8167f12be74330c1b881b5aa3df95f5bd66d26f42cc03b87a7c38946c571
adhd@adhd:/opt/cowrie/var/lib/cowrie/downloads$ md5sum
^C
adhd@adhd:/opt/cowrie/var/lib/cowrie/downloads$ md5sum f14e8167f12be74330c1b881b5aa3df95f5bd66d26f42cc03b87a7c38946c571
e2620d4a5a0f8d80dd4b16de59af981f f14e8167f12be74330c1b881b5aa3df95f5bd66d26f42cc03b87a7c38946c571
adhd@adhd:/opt/cowrie/var/lib/cowrie/downloads$
```

the MD5 checksum matches the original file on the Host VM. Since the checksums match, it means Cowrie captured an exact copy of the file.

This explain that Cowrie's behavior is designed to log and retain all files downloaded by attackers, allowing for post-attack analysis without compromising the actual system.

Task 7 Exit from cowrie ssh session:

```
cyuser@kali:~$ ssh root@10.10.1.6 -p 2222
root@10.10.1.6's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# exit
Connection to server closed.
root@localhost:~#
```

```
adhd@adhd:~$ tail -f /opt/cowrie/var/log/cowrie/cowrie.log
2023-10-25T06:25:15.102876Z [SSHSservice b'ssh-userauth' on HoneyPotSSHTransport,54,10.10.1.4] starting s
ervice b'ssh-connection'
2023-10-25T06:25:15.104335Z [SSHSservice b'ssh-connection' on HoneyPotSSHTransport,54,10.10.1.4] got chan
nel b'session' request
2023-10-25T06:25:15.104768Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] channel open
2023-10-25T06:25:15.104992Z [SSHSservice b'ssh-connection' on HoneyPotSSHTransport,54,10.10.1.4] got glob
al b'no-more-sessions@openssh.com' request
2023-10-25T06:25:15.192295Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] pty request: b'xterm-256color' (41, 123, 0, 0)
2023-10-25T06:25:15.192464Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] Terminal Size: 123 41
2023-10-25T06:25:15.193278Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] request_env: b'LANG'=b'en_US.utf8'
2023-10-25T06:25:15.193717Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] getting shell
2023-10-25T06:25:20.054115Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] CMD: exit
2023-10-25T06:25:20.055763Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] Command found: exit
2023-10-25T06:28:10.799168Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] CMD: ls
2023-10-25T06:28:10.803693Z [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransp
ort,54,10.10.1.4] Command found: ls
```

This log shows the exit command but when we exit the ssh from the attack vm seen in the first screenshot we see it doesn't fully exit and to test it we type a command here I used a simple ls and then we check the cowrie logs on our adhd vm and we see in the second screenshot that yes it was captured in the logs which can indicate that you haven't fully exited the honeypot, even after running exit.

Task 8 SSH out of cowrie:

```
cyuser@kali:~$ pwd
/home/cyuser
cyuser@kali:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```



```

cyuser@kali:~$ ssh root@10.10.1.6 -p 2222
root@10.10.1.6's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# ssh cyuser@10.10.1.5
The authenticity of host '10.10.1.5 (10.10.1.5)' can't be established.
RSA key fingerprint is 9d:30:97:8a:9e:48:0d:de:04:8d:76:3a:7b:4b:30:f8.
Are you sure you want to continue connecting (yes/no)? y
Warning: Permanently added '10.10.1.5' (RSA) to the list of known hosts.
cyuser@10.10.1.5's password:
Linux localhost 2.6.26-2-686 #1 SMP Wed Nov 4 20:45:37 UTC 2009 i686
Last login: Mon Oct 23 20:23:10 2023 from 192.168.9.4
root@localhost:~# pwd
/root
root@localhost:~# ls

```

```

cyuser@kali: ~
File Actions Edit View Help
Oct 25 08:02:41 kali sudo: cyuser : TTY=pts/0 ; PWD=/home/cyuser ; USER=r
oot ; COMMAND=/usr/sbin/service apache2 start
Oct 25 08:02:41 kali sudo: pam_unix(sudo:session): session opened for user
root by (uid=0)
Oct 25 08:02:43 kali sudo: pam_unix(sudo:session): session closed for user
root
Oct 25 08:05:01 kali CRON[2308]: pam_unix(cron:session): session opened for
user root by (uid=0)
Oct 25 08:05:01 kali CRON[2308]: pam_unix(cron:session): session closed for
user root
Oct 25 08:09:01 kali CRON[2313]: pam_unix(cron:session): session opened for
user root by (uid=0)
Oct 25 08:09:01 kali CRON[2313]: pam_unix(cron:session): session closed for
user root
Oct 25 08:15:01 kali CRON[2370]: pam_unix(cron:session): session opened for
user root by (uid=0)
Oct 25 08:15:01 kali CRON[2370]: pam_unix(cron:session): session closed for
user root
Oct 25 08:17:01 kali CRON[2374]: pam_unix(cron:session): session opened for
user root by (uid=0)
Oct 25 08:17:01 kali CRON[2374]: pam_unix(cron:session): session closed for
user root
Oct 25 08:25:01 kali CRON[2382]: pam_unix(cron:session): session opened for
user root by (uid=0)
Oct 25 08:25:01 kali CRON[2382]: pam_unix(cron:session): session closed for
user root
Oct 25 08:34:54 kali sudo: pam_unix(sudo:auth): Couldn't open /etc/securett
y: No such file or directory
Oct 25 08:35:00 kali sudo: pam_unix(sudo:auth): Couldn't open /etc/securett
y: No such file or directory
Oct 25 08:35:00 kali sudo: cyuser : TTY=pts/0 ; PWD=/home/cyuser ; USER=r
oot ; COMMAND=/usr/bin/cat /var/log/auth.log
cyuser@kali:~$

```

Task 9 Cowrie Files:

Using the command “cat /opt/cowrie/etc/userdb.example” we can view the userdb.example file and this file contains a list of usernames and associated passwords which cowrie uses to simulate login attempts this is so this file can mimic real user accounts with passwords enticing attackers to believe they have gained access. Here is the screenshot of this:

```

adh@adh:~$ cat /opt/cowrie/etc/userdb.example
# Example userdb.txt
# This file may be copied to etc/userdb.txt.
# If etc/userdb.txt is not present, built-in defaults will be used.
#
# ':' separated fields, file is processed line for line
# processing will stop on first match
#
# Field #1 contains the username
# Field #2 is currently unused
# Field #3 contains the password
# '*' for password allows any password
# '!' at the start of a password will not grant this password access
# '/' can be used to write a regular expression
#
root:x:!root
root:x:!123456
root:x:!/honeypot/i
root:x:*
tomcat:x:*
oracle:x:*

```

The passwd file in cowrie's fake file system mimics the /etc/passwd file on a real system. Cowrie simulates a realistic list of user accounts to deceive attackers when they attempt to access user information. The command for this is “cat /opt/cowrie/honeyfs/etc/passwd” Here is the screenshot of this:

```

adh@adh:~$ cat /opt/cowrie/honeyfs/etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
richard:x:1000:1000:Richard Texas,,,:/home/richard:/bin/bash

```

```

adh@adh:~$ cat /opt/cowrie/honeyfs/etc/hostname
svr04

```

```

adhd@adhd:~$ cat /opt/cowrie/honeyfs/proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 23
model name     : Intel(R) Core(TM)2 Duo CPU      E8200   @ 2.66GHz
stepping       : 6
cpu MHz        : 2133.304
cache size     : 6144 KB
physical id    : 0
siblings       : 2
core id        : 0
cpu cores      : 2
apicid         : 0
initial apicid : 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 10
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts a
pi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni monitor
ds_cpl vmx smx est tm2 ssse3 cx16 xtpr sse4_1 lahf_lm
bogomips       : 4270.03
clflush size   : 64
cache_alignmen : 64
address sizes   : 36 bits physical, 48 bits virtual
power management:

processor       : 1
vendor_id      : GenuineIntel
cpu family     : 6
model          : 23
model name     : Intel(R) Core(TM)2 Duo CPU      E8200   @ 2.66GHz
stepping       : 6
cpu MHz        : 2133.304
cache size     : 6144 KB
physical id    : 0

```

```

siblings       : 2
core id        : 1
cpu cores      : 2
apicid         : 1
initial apicid : 1
fpu            : yes
fpu_exception  : yes
cpuid level    : 10
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts ac
pi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni monitor
ds_cpl vmx smx est tm2 ssse3 cx16 xtpr sse4_1 lahf_lm
bogomips       : 4266.61
clflush size   : 64
cache_alignmen : 64
address sizes   : 36 bits physical, 48 bits virtual
power management:

```

Lab 6 NetFlow:

Nfdump -R netflow -s srcp/packets/bytes

Nfdump -R netflow -s dstp/packets/bytes

Nfdump -R netflow -s dstip/packets/bytes

nfdump -R netflow -A proto

Nfdump -R netflow -s port/packet

1. What is the IP address of the target? *Hint*: look for a disproportionately large amount of traffic hitting a single IP address.

```
securityonion@so:~/netflows$ nfdump -R netflow -s dstip/packets/bytes
Top 10 Dst IP Addr ordered by packets:
Date first seen      Duration Proto      Dst IP Addr      Flows(%)      Packets(%)      Bytes(%)
pps      bps      bpp
2007-02-23 18:59:03.544 4396582.231 any      195.88.49.121    29.4 M(95.9)    3.2 G(99.3)    93.3 G(7
9.6)      728      169749    29
2007-02-23 18:58:19.163 4392255.301 any      195.88.49.135    64124( 0.2)    2.2 M( 0.1)    2.8 G(
2.4)      0        5034     1280
2007-02-23 18:59:43.158 4384832.859 any      195.88.49.125    41148( 0.1)    584584( 0.0)    318.5 M(
0.3)      0        581      544
2007-02-23 19:02:26.557 4377623.730 any      195.88.49.34     5353( 0.0)    382984( 0.0)    291.8 M(
0.2)      0        533      761
2007-02-23 18:59:06.615 4410239.857 any      195.88.49.97     158771( 0.5)    376831( 0.0)    57.4 M(
0.0)      0        104     152
2007-02-23 19:00:03.097 4390002.747 any      195.88.49.114    12523( 0.0)    264807( 0.0)    30.4 M(
0.0)      0        55       114
2007-02-24 16:26:47.147 4313941.168 any      212.248.249.53   10169( 0.0)    243925( 0.0)    306.7 M(
0.3)      0        568     1257
2007-02-23 19:00:43.072 4298738.686 any      42.160.11.92     1669( 0.0)    220923( 0.0)    13.6 M(
0.0)      0        25       61
2007-02-23 19:23:16.020 4383179.412 any      193.1.50.227     1619( 0.0)    178453( 0.0)    10.8 M(
0.0)      0        19       60
2007-02-23 19:00:08.322 4383868.113 any      195.88.49.129    54853( 0.2)    167672( 0.0)    21.2 M(
0.0)      0        38      126
```

The ip address of target is 195.88.49.121 and to get this first we went into the netflows directory and then we entered the command 'nfdump -R netflow -s dstip/packets/bytes' and here it displayed the bytes which is the sixth column and we see that the first entry has 93.3 gb it also has a high flow % showing it is dominating the traffic.

2. What protocol was used in the attack?

```
securityonion@so:~/netflows$ nfdump -R netflow -s proto
Top 10 Protocol ordered by flows:
Date first seen      Duration Proto      Protocol      Flows(%)      Packets(%)
pps      bps      bpp
2007-02-23 18:59:06.535 4410239.937 UDP           17      29.8 M(97.5)    3.2 G(
9.2)      724      168511    29
2007-02-23 18:58:08.402 4403815.123 TCP           6       748480( 2.4)    29.5 M(
0.7)      6       44194     825
2007-02-23 19:00:55.526 4390867.484 ICMP          1       21814( 0.1)    52007(
0.0)      0        6        70
2007-02-23 19:03:47.116 111880.896 ESP          50       218( 0.0)    30835(
.0)      0       1416     642
2007-02-24 03:05:10.329 85770.034 RSVP          46       17( 0.0)    25(
0)      0        0      224

Summary: total flows: 30615402, total bytes: 117.2 G, total packets: 3.2 G, avg bps
: 731, avg bpp: 36
Time window: 2007-02-23 18:58:08 - 2007-04-15 20:03:06
Total flows processed: 30615402, Blocks skipped: 0, Bytes read: 1592026680
Sys: 2.808s flows/second: 10902920.9 Wall: 4.379s flows/second: 6990108.3
securityonion@so:~/netflows$
```

The protocol used was UDP as it dominates the traffic with 3.2 gb of packets and close to 30 mb of flow making up 97% of the total flows

- Looking at the file sizes of the netflows (ls -lh), estimate the start time and end time of the attack. **Note:** don't forget to account for the one hour offset between the times indicated by the nfcapd filenames and the actual flow times.

```
-rw-r--r-- 1 securityunion securityunion 125K Nov 21 2016 nfcapd.200702240320
-rw-r--r-- 1 securityunion securityunion 67K Nov 21 2016 nfcapd.200702240325
-rw-r--r-- 1 securityunion securityunion 88K Nov 21 2016 nfcapd.200702240330
-rw-r--r-- 1 securityunion securityunion 115K Nov 21 2016 nfcapd.200702240335
-rw-r--r-- 1 securityunion securityunion 136K Nov 21 2016 nfcapd.200702240340
-rw-r--r-- 1 securityunion securityunion 124K Nov 21 2016 nfcapd.200702240345
-rw-r--r-- 1 securityunion securityunion 102K Nov 21 2016 nfcapd.200702240350
-rw-r--r-- 1 securityunion securityunion 88K Nov 21 2016 nfcapd.200702240355
-rw-r--r-- 1 securityunion securityunion 3.9M Nov 21 2016 nfcapd.200702240400
-rw-r--r-- 1 securityunion securityunion 11M Nov 21 2016 nfcapd.200702240405
-rw-r--r-- 1 securityunion securityunion 13M Nov 21 2016 nfcapd.200702240410
-rw-r--r-- 1 securityunion securityunion 17M Nov 21 2016 nfcapd.200702240415
-rw-r--r-- 1 securityunion securityunion 11M Nov 21 2016 nfcapd.200702240420
-rw-r--r-- 1 securityunion securityunion 29M Nov 21 2016 nfcapd.200702240425
-rw-r--r-- 1 securityunion securityunion 27M Nov 21 2016 nfcapd.200702240430
-rw-r--r-- 1 securityunion securityunion 16M Nov 21 2016 nfcapd.200702240435
-rw-r--r-- 1 securityunion securityunion 18M Nov 21 2016 nfcapd.200702240440
-rw-r--r-- 1 securityunion securityunion 14M Nov 21 2016 nfcapd.200702240445
-rw-r--r-- 1 securityunion securityunion 17M Nov 21 2016 nfcapd.200702240450
-rw-r--r-- 1 securityunion securityunion 19M Nov 21 2016 nfcapd.200702240455
-rw-r--r-- 1 securityunion securityunion 15M Nov 21 2016 nfcapd.200702240500
-rw-r--r-- 1 securityunion securityunion 15M Nov 21 2016 nfcapd.200702240505
-rw-r--r-- 1 securityunion securityunion 15M Nov 21 2016 nfcapd.200702240510
-rw-r--r-- 1 securityunion securityunion 15M Nov 21 2016 nfcapd.200702240515
-rw-r--r-- 1 securityunion securityunion 18M Nov 21 2016 nfcapd.200702240520
-rw-r--r-- 1 securityunion securityunion 16M Nov 21 2016 nfcapd.200702240525
-rw-r--r-- 1 securityunion securityunion 23M Nov 21 2016 nfcapd.200702240530
```

When we use the command ls -lh we get this output shown above and from here we must find the start and end date of the attack so here we see that the start date starts at the 88k file size mark and the date of that one is 2007/02/24 03:55 but because we have to account for the hour setback it is 2007/02/24 02:55 and as for the end date we see that the file sizes aren't as big and start to decrease to the original sizes here

```
-rw-r--r-- 1 securityunion securityunion 2.4M Nov 21 2016 nfcapd.200702241505
-rw-r--r-- 1 securityunion securityunion 2.3M Nov 21 2016 nfcapd.200702241510
-rw-r--r-- 1 securityunion securityunion 2.3M Nov 21 2016 nfcapd.200702241515
-rw-r--r-- 1 securityunion securityunion 2.2M Nov 21 2016 nfcapd.200702241520
-rw-r--r-- 1 securityunion securityunion 2.3M Nov 21 2016 nfcapd.200702241525
-rw-r--r-- 1 securityunion securityunion 2.3M Nov 21 2016 nfcapd.200702241530
-rw-r--r-- 1 securityunion securityunion 2.4M Nov 21 2016 nfcapd.200702241535
-rw-r--r-- 1 securityunion securityunion 2.4M Nov 21 2016 nfcapd.200702241540
-rw-r--r-- 1 securityunion securityunion 2.4M Nov 21 2016 nfcapd.200702241545
-rw-r--r-- 1 securityunion securityunion 1.1M Nov 21 2016 nfcapd.200702241550
-rw-r--r-- 1 securityunion securityunion 283K Nov 21 2016 nfcapd.200702241555
-rw-r--r-- 1 securityunion securityunion 1.2M Nov 21 2016 nfcapd.200702241600
-rw-r--r-- 1 securityunion securityunion 2.8M Nov 21 2016 nfcapd.200702241605
-rw-r--r-- 1 securityunion securityunion 250K Nov 21 2016 nfcapd.200702241610
-rw-r--r-- 1 securityunion securityunion 245K Nov 21 2016 nfcapd.200702241615
```

At the file size 250k the time here is 2007/02/24 16:10 which for the hour setback is 15:10.

4. Analyse the traffic (port and protocol) to/from the victim before the attack began.
What does this tell you about the victim?

```

2007-02-23 20:09:58.946 0.000 TCP 195.39.83.112:53646 -> 195.88.49.1
21:80 1 40 1
2007-02-23 20:09:02.373 29.462 TCP 195.88.49.121:80 -> 40.110.57.
32:2548 26 7786 1
2007-02-23 20:09:17.772 18.329 TCP 195.88.49.121:80 -> 44.5.161.1
88:1315 11 2064 1
2007-02-23 20:09:26.583 19.055 TCP 195.88.49.121:80 -> 193.144.11.1
33:3172 59 72629 1
2007-02-23 20:09:39.387 3.787 TCP 44.13.100.5:59989 -> 195.88.49.1
21:80 23 6575 1
2007-02-23 20:09:27.386 20.198 TCP 44.29.95.93:56436 -> 195.88.49.1
21:80 21 1528 1
2007-02-23 20:09:25.770 31.677 TCP 44.19.6.1:3873 -> 195.88.49.1
21:80 31 4312 1
2007-02-23 20:09:38.465 17.412 TCP 195.88.49.121:80 -> 32.168.190.2
50:56536 19 22458 1
2007-02-23 20:09:39.628 20.752 TCP 195.88.49.121:80 -> 44.13.100
.5:59995 20 20754 1
Summary: total flows: 4087, total bytes: 234.5 M, total packets: 253872, avg bps
: 3.1 M, avg pps: 422, avg bpp: 923
Time window: 2007-02-23 18:58:08 - 2007-04-14 13:22:29
Total flows processed: 103004, Blocks skipped: 0, Bytes read: 5356400
Sys: 0.436s flows/second: 236247.7 Wall: 2.548s flows/second: 40417.6
securityonion@so:~/netflows$

```

```

securityonion@so:~/netflows$ nfdump -R netflow -t '2007/02/23.23:00-2007/02/24.00:55' -s dstport 'd
t ip 195.88.49.121'
Top 10 Dst Port ordered by flows:
Date first seen Duration Proto Dst Port Flows(%) Packets(%) Bytes(%)
pps bps bpp
2007-02-23 23:00:02.619 6897.505 any 80 4684(98.0) 183908(99.9) 16.8 M(99
8) 26 19428 91
2007-02-23 23:01:33.447 6579.087 any 1026 45( 0.9) 45( 0.0) 22651( 0
1) 0 27 503
2007-02-23 23:03:28.604 6567.376 any 1027 24( 0.5) 24( 0.0) 12423( 0
1) 0 15 517
2007-02-23 23:21:15.365 4279.638 any 0 16( 0.3) 41( 0.0) 2308( 0
0) 0 4 56
2007-02-23 23:08:23.448 3744.706 any 53 5( 0.1) 30( 0.0) 1771( 0
0) 0 3 59
2007-02-23 23:16:57.021 127.073 any 5269 2( 0.0) 8( 0.0) 352( 0
0) 0 22 44
2007-02-24 00:48:19.023 0.000 any 4306 1( 0.0) 1( 0.0) 40( 0
0) 0 0 40
2007-02-23 23:31:33.219 0.000 any 1434 1( 0.0) 1( 0.0) 404( 0
0) 0 0 404
2007-02-23 23:13:49.874 0.000 any 135 1( 0.0) 1( 0.0) 48( 0
0) 0 0 48
2007-02-24 00:47:43.621 0.000 any 2967 1( 0.0) 1( 0.0) 48( 0
0) 0 0 48
Summary: total flows: 4781, total bytes: 16.8 M, total packets: 184061, avg bps: 19475, avg pps: 26
, avg bpp: 91
Time window: 2007-02-23 18:58:08 - 2007-04-14 17:58:05
Total flows processed: 255070, Blocks skipped: 0, Bytes read: 13264384
Sys: 0.020s flows/second: 12753500.0 Wall: 0.028s flows/second: 9023915.7
securityonion@so:~/netflows$

```

```

securityonion@so:~/netflows$ nfdump -R netflow -t '2007/02/23.23:00-2007/02/24.00:55' -A proto 'ip
95.88.49.121'
Date flow start Duration Proto Packets Bytes bps Bpp Flows
2007-02-23 23:21:15.365 4279.638 ICMP 41 2308 4 56 16
2007-02-23 23:01:33.447 6682.533 UDP 101 37327 44 369 76
2007-02-23 23:00:02.619 6897.508 TCP 499425 422.5 M 489990 845 10703
Summary: total flows: 10795, total bytes: 422.5 M, total packets: 499567, avg bps: 490036, avg pps:
72, avg bpp: 845
Time window: 2007-02-23 18:58:08 - 2007-04-14 17:58:05
Total flows processed: 255070, Blocks skipped: 0, Bytes read: 13264384
Sys: 0.024s flows/second: 10627916.7 Wall: 0.033s flows/second: 7531742.8
securityonion@so:~/netflows$

```


- For this part the majority of the traffic to the victim was directed at TCP port 80, which is typically associated with HTTP web traffic. This suggests that the victim was likely a web server or hosted web services, as it was handling a significant amount of incoming HTTP requests.
- There were also minor flows on ports 1026 and 1027, which may indicate background services or unrelated communication that the server occasionally handled.
- ICMP, UDP, and TCP protocols were all observed in the traffic, but TCP accounted for the majority of the data volume and flows, especially on port 80, reinforcing the idea that HTTP traffic was dominant.

Traffic Volume and Behavior:

- Before the attack, the server experienced substantial TCP traffic with high packet and byte counts, especially on port 80. This indicates that the server was handling a regular load of web traffic, which could be typical for a public-facing web server.
- The presence of ICMP traffic could suggest periodic network monitoring or diagnostics, which might be typical for a server that is routinely monitored for availability or performance.

Interpretation of Victim's Role:

- Given the high volume of HTTP traffic on port 80, it is likely that the victim was functioning as a web server, hosting online services that attracted regular traffic. This role makes it a potential target for DDoS attacks aimed at overwhelming its ability to serve legitimate requests.
- The combination of multiple protocols and ports, albeit with a focus on HTTP, suggests the victim was accessible for various types of communication, which attackers might exploit to disrupt its services.

5. What ports were targeted during the attack? And 6.

```
securityonion@so:~/netflows$ nfdump -R netflow -t '2007/02/24.07:00-2007/02/24.08:00' -A proto 'ip 95.88.49.121'
Date flow start      Duration  Proto  Packets  Bytes      bps      Bpp  Flows
2007-02-24 07:00:51.557 3512.593 ICMP    47      15154      34       322    30
2007-02-24 07:00:00.000 3600.998 UDP     368.4 M  10.7 G     23.7 M    29    3270683
2007-02-24 07:00:00.187 3600.599 TCP     30333   11.9 M     26461     392    6534
Summary: total flows: 3277247, total bytes: 10.7 G, total packets: 368.4 M, avg bps: 23.8 M, avg pp
: 102314, avg bpp: 29
Time window: 2007-02-23 18:58:08 - 2007-04-15 01:05:02
Total flows processed: 18504077, Blocks skipped: 0, Bytes read: 962226452
Sys: 1.668s flows/second: 11093571.3 Wall: 4.379s flows/second: 4224746.5
```

```

securityonion@so:~/netflows$ nfdump -R netflow -t '2007/02/24.07:00-2007/02/24.08:00' -s dstport/pa
kets 'dst ip 195.88.49.121'
Top 10 Dst Port ordered by packets:
Date first seen      Duration Proto      Dst Port    Flows(%)    Packets(%)    Bytes(%)
pps      bps      bpp
2007-02-24 07:00:01.015 3599.771 any      80      3883( 0.1)    26143( 0.0)    2.5 M( 0
0)      7      5618      96
2007-02-24 07:00:35.160 3535.734 any     19450      104( 0.0)    13607( 0.0)    394603( 0
0)      3      892      29
2007-02-24 07:00:13.130 3578.402 any     29529      101( 0.0)    13414( 0.0)    389006( 0
0)      3      869      29
2007-02-24 07:00:35.555 3535.120 any      423      109( 0.0)    13204( 0.0)    382916( 0
0)      3      866      29
2007-02-24 07:00:35.455 3493.588 any     45868      102( 0.0)    13048( 0.0)    378392( 0
0)      3      866      29
2007-02-24 07:00:35.508 3535.275 any     9520      110( 0.0)    13006( 0.0)    377174( 0
0)      3      853      29
2007-02-24 07:00:13.797 3557.247 any     29735      107( 0.0)    12983( 0.0)    376507( 0
0)      3      846      29
2007-02-24 07:00:35.707 3545.936 any     37366      107( 0.0)    12972( 0.0)    376188( 0
0)      3      848      29
2007-02-24 07:00:30.707 3546.545 any     17365      105( 0.0)    12947( 0.0)    375463( 0
0)      3      846      29
2007-02-24 07:00:35.597 3535.234 any     45950       97( 0.0)    12876( 0.0)    373404( 0
0)      3      844      29

Summary: total flows: 3274535, total bytes: 10.7 G, total packets: 368.4 M, avg bps: 23.7 M, avg pp
: 102311, avg bpp: 29
Time window: 2007-02-23 18:58:08 - 2007-04-15 01:05:02
Total flows processed: 18504077, Blocks skipped: 0, Bytes read: 962226452
Sys: 1.860s flows/second: 9948428.5 Wall: 3.032s flows/second: 6101901.4

```

As we can see here during the attack the attack appears to target both a specific service (HTTP on port 80) and random high-numbered ports, likely to overwhelm the server with UDP or mixed traffic.

6. What type of attack was this? Briefly outline how this attack operates.

This attack from what I have analyzed of it seems to be a Multi-vector DDoS attack. How this attack operates is like this:

- Botnet Deployment:
 - The attacker likely used a botnet (a network of infected devices) to generate traffic.
- Overwhelming the Target:
 - The HTTP flood targets application-level resources (e.g., web servers), while the UDP and ICMP floods exhaust bandwidth and processing power.
- Denial of Service:
 - The combined volume of traffic overwhelms the victim's resources, leading to service degradation or a complete crash.

And from what I observed of this attack is the following:

- Application-Layer Flood: Targeting HTTP service (port 80).
- Volumetric UDP Flood: Targeting random high-numbered UDP ports.
- Potential ICMP Flood: To exhaust bandwidth and processing resources.

7. What are the IP addresses of the attackers? Verify that each IP address identified is part of the attack by analysing the traffic from that address to the victim. Does this traffic correspond with your attack hypothesis?

```
securityonion@so:~/netflows$ nfdump -R netflow -s srcip/packets 'dst ip 195.88.49.121'
Top 10 Src IP Addr ordered by packets:
Date first seen      Duration Proto      Src IP Addr      Flows(%)      Packets(%)      Bytes(%)
pps      bps      bpp
2007-02-24 03:02:25.980 4311370.089 any      213.63.169.117    3.8 M(13.1)    836.4 M(26.1)    24.3 G(
6.0)      193      45007      29
2007-02-24 02:59:59.672 4319571.482 any      33.106.25.243     5.6 M(19.2)    830.3 M(25.9)    24.1 G(
5.8)      192      44595      29
2007-02-24 03:05:15.668 4312716.729 any      33.106.23.177     2.8 M( 9.6)    736.8 M(23.0)    21.4 G(
2.9)      170      39638      29
2007-02-24 03:26:02.871 4318008.723 any      207.39.221.61     5.7 M(19.3)    642.2 M(20.1)    18.6 G(
0.0)      148      34507      29
2007-02-24 09:40:26.891 4314609.658 any      33.51.2.133       3.6 M(12.1)    124.3 M( 3.9)    3.6 G(
3.9)      28       6681      29
2007-02-24 03:07:41.797 4319103.886 any      43.170.142.79     5.1 M(17.4)    14.5 M( 0.5)    421.4 M(
0.5)      3        780      29
2007-02-24 09:45:12.053 4313382.017 any      57.119.85.138     2.5 M( 8.6)    8.2 M( 0.3)    239.1 M(
0.3)      1        443      29
2007-02-24 16:26:47.147 4313980.435 any      212.248.249.53    10292( 0.0)    231893( 0.0)    12.4 M(
0.0)      0        23       53
2007-02-23 19:00:12.991 5339.708 any      44.19.6.1         46( 0.0)      49919( 0.0)     2.1 M( 0
0)      9        3194     42
2007-02-24 13:59:16.394 4296293.282 any      44.21.171.81      192( 0.0)     40916( 0.0)     2.2 M(
0.0)      0        4        53

Summary: total flows: 29371456, total bytes: 93.3 G, total packets: 3.2 G, avg bps: 169749, avg pps
728, avg bpp: 29
Time window: 2007-02-23 18:58:08 - 2007-04-15 20:03:06
Total flows processed: 30615402, Blocks skipped: 0, Bytes read: 1592026680
Sys: 3.412s flows/second: 8972861.1 Wall: 4.396s flows/second: 6963273.1
```

Using the command “nfdump -R netflow -s srcip/packets 'dst ip 195.88.49.121'”:

Primary Attacker IPs:

- 213.63.169.117
- 33.106.25.243
- 33.106.23.177
- 207.39.221.61
- 33.51.2.133

8. Suggest one possible method of mitigating against this attack.

- Rate Limiting and Connection Filtering:

The service applies rate limiting to limit requests per IP address, which can mitigate HTTP flood attacks. UDP floods targeting random high ports can be filtered by blocking unused ports or limiting UDP traffic rates.

- Protocol-Specific Mitigation:
For TCP floods: Use SYN flood protection mechanisms (e.g., SYN cookies).
For UDP floods: Drop packets targeting unused or uncommon ports. For ICMP floods: Limit or block ICMP requests.
- Real-Time Monitoring:
DDoS protection services provide dashboards for real-time monitoring of traffic patterns, enabling early detection of ongoing attacks.

Lab 8 PCAP: Drive by Download

Part 1 Drive by Download Without FAST FLUX:

DNS Server and Client Host IPs

- DNS Server: 10.0.0.2
- Client Host: 10.0.0.130

Dns queries:

Response queries:

Filter:		dns.flags.response == 1		Expression...		Clear	Apply	Save
No.	Time	Source	Destination	Protocol	Length	Info		
4	3.45	10.0.0	10.0.0.130	DNS	276	Standard query response 0x0453 CNAME melkor.nask.waw.pl A 195.187.7.66		
39	3.60	10.0.0	10.0.0.130	DNS	271	Standard query response 0x7050 CNAME urs.microsoft.com.nsadc.net A 213.199.1		
164	5.71	10.0.0	10.0.0.130	DNS	214	Standard query response 0xf53c CNAME homebank.pl A 212.85.111.79		
179	6.60	10.0.0	10.0.0.130	DNS	271	Standard query response 0x8e3e A 212.160.67.149		
186	6.86	10.0.0	10.0.0.130	DNS	122	Standard query response 0x8a3f A 85.255.120.194		
199	7.22	10.0.0	10.0.0.130	DNS	134	Standard query response 0x723e A 211.95.72.85		
203	7.32	10.0.0	10.0.0.130	DNS	258	Standard query response 0x0b00 A 66.232.114.139		
215	7.66	10.0.0	10.0.0.130	DNS	355	Standard query response 0x1156 A 193.59.201.62		
579	10.4	10.0.0	10.0.0.130	DNS	105	Standard query response 0xf53c CNAME homebank.pl A 212.85.111.79		
809	14.9	10.0.0	10.0.0.130	DNS	130	Standard query response 0x2c02 No such name		
811	14.9	10.0.0	10.0.0.130	DNS	160	Standard query response 0x4e04 No such name		

Queries only:

Filter:		dns.flags.response == 0		▼	Expression...	Clear	Apply	Save
No.	Time	Source	Destination	Protocol	Length	Info		
3	3.44	10.0.0	10.0.0.2	DNS	71	Standard query 0x0453	A	www.cert.pl
27	3.56	10.0.0	10.0.0.2	DNS	77	Standard query 0x7050	A	urs.microsoft.com
162	4.56	10.0.0	10.0.0.2	DNS	75	Standard query 0xf53c	A	www.homebank.pl
163	5.55	10.0.0	10.0.0.2	DNS	75	Standard query 0xf53c	A	www.homebank.pl
177	6.58	10.0.0	10.0.0.2	DNS	71	Standard query 0x723e	A	lsense.info
178	6.59	10.0.0	10.0.0.2	DNS	76	Standard query 0x8e3e	A	stat.webmedia.pl
181	6.62	10.0.0	10.0.0.2	DNS	70	Standard query 0x8a3f	A	winhex.org
202	7.32	10.0.0	10.0.0.2	DNS	69	Standard query 0x0b00	A	jezl0.com
214	7.66	10.0.0	10.0.0.2	DNS	71	Standard query 0x1156	A	www.nask.pl
808	14.9	10.0.0	10.0.0.2	DNS	73	Standard query 0x2c02	A	www.koszty.pl
810	14.9	10.0.0	10.0.0.2	DNS	85	Standard query 0x4e04	A	www.koszty.pl.localdomain

The IP addresses of the 3 benign websites are.

- www.cert.pl: 195.187.7.66 – as we see in the screenshot response to 0x0453 which is www.cert.pl.
- www.nask.pl: 193.59.201.62-as we can see in the screenshot response to 0x1156 which is www.nask.pl.
- urs.microsoft.com: 213.199.161.251- as we can see in the screenshot response to 0x7050 which is urs.microsoft.com.

Now filtering out the benign websites we use the command `!(ip.addr == 195.187.7.66 or ip.addr == 213.199.161.251 or ip.addr == 193.59.201.62)` and this will filter out all of the benign.

Filter:		addr == 213.199.161.251 or ip.addr == 193.59.201.62)		▼	Expression...	Clear	Apply	Save
No.	Time	Source	Destination	Protocol	Length	Info		
181	6.62	10.0.0.130	10.0.0.2	DNS	70	Standard query 0x8a3f	A	winhex.org
182	6.66	212.160.67	10.0.0.130	TCP	58	http > bvtsonar [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460		
183	6.66	212.85.111	10.0.0.130	HTTP	646	[TCP Retransmission] HTTP/1.1 200 OK (text/css)		
184	6.66	10.0.0.130	212.160.67	TCP	54	bvtsonar > http [ACK] Seq=1 Ack=1 Win=65535 Len=0		
185	6.66	10.0.0.130	212.85.111	TCP	54	elfiq-repl > http [ACK] Seq=451 Ack=2774 Win=64943 Len=0		
186	6.86	10.0.0.2	10.0.0.130	DNS	122	Standard query response 0x8a3f	A	85.255.120.194
187	6.87	10.0.0.130	85.255.120	TCP	62	blaze > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1		
188	6.87	10.0.0.130	212.160.67	HTTP	317	GET /cgi-bin/stat?homebank&stat4u HTTP/1.1		
189	6.87	212.160.67	10.0.0.130	TCP	54	http > bvtsonar [ACK] Seq=1 Ack=264 Win=64240 Len=0		
190	6.92	212.160.67	10.0.0.130	HTTP	1212	HTTP/1.1 200 OK (GIF87a) (GIF87a) (image/gif)		
191	6.92	212.160.67	10.0.0.130	TCP	54	http > bvtsonar [FIN, PSH, ACK] Seq=1159 Ack=264 Win=64240 Len=0		
192	6.93	10.0.0.130	212.160.67	TCP	54	bvtsonar > http [ACK] Seq=264 Ack=1160 Win=64377 Len=0		
193	6.93	10.0.0.130	212.160.67	TCP	54	bvtsonar > http [FIN, ACK] Seq=264 Ack=1160 Win=64377 Len=0		
194	6.93	212.160.67	10.0.0.130	TCP	54	http > bvtsonar [ACK] Seq=1160 Ack=265 Win=64239 Len=0		
195	7.07	85.255.120	10.0.0.130	TCP	58	http > blaze [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460		
196	7.07	10.0.0.130	85.255.120	TCP	54	blaze > http [ACK] Seq=1 Ack=1 Win=65535 Len=0		
197	7.07	10.0.0.130	85.255.120	HTTP	379	GET /tds/in.cgi?3 HTTP/1.1		
198	7.07	85.255.120	10.0.0.130	TCP	54	http > blaze [ACK] Seq=1 Ack=326 Win=64240 Len=0		
199	7.22	10.0.0.2	10.0.0.130	DNS	134	Standard query response 0x723e	A	211.95.72.85
200	7.22	10.0.0.130	211.95.72.8	TCP	62	unizensus > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1		
201	7.29	85.255.120	10.0.0.130	HTTP	596	HTTP/1.1 302 Found (text/html)		
202	7.32	10.0.0.130	10.0.0.2	DNS	69	Standard query 0x0b00	A	jezl0.com

Now we see what other hosts the client attempted to connect to:

```
!(ip.addr == 195.187.7.66 or ip.addr == 213.199.161.251 or ip.addr == 193.59.201.62) && ip.src == 10.0.0.130 && tcp.flags.syn == 2
```

In this command we are still filtering out the benign websites and we are filtering for tcp and from the source ip “10.0.0.130” of the client and we will look for all the syn packets sent from the host

Filter: .201.62) && ip.src == 10.0.0.130 && tcp.flags.syn == 2 Expression... Clear Apply Save						
No.	Time	Source	Destination	Protocol	Length	Info
165	5.72	10.0.0.130	212.85.111.79	TCP	62	elfiq-repl > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
180	6.62	10.0.0.130	212.160.67.149	TCP	62	bvtsonar > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
187	6.87	10.0.0.130	85.255.120.194	TCP	62	blaze > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
200	7.22	10.0.0.130	211.95.72.85	TCP	62	unizensus > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
204	7.32	10.0.0.130	66.232.114.139	TCP	62	winpoplanmess > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
447	8.57	10.0.0.130	72.36.162.50	TCP	62	cl222-acse > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
606	11.0	10.0.0.130	208.72.169.133	TCP	62	resacommunity > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1
776	14.0	10.0.0.130	208.72.169.133	TCP	62	[TCP Retransmission] resacommunity > http [SYN] Seq=0 Win=65535 Len=0
818	20.0	10.0.0.130	208.72.169.133	TCP	62	[TCP Retransmission] resacommunity > http [SYN] Seq=0 Win=65535 Len=0

Above is the filtered information.

The other host addresses are as seen from the screenshot above are:

- 212.85.111.79
- 212.160.67.149
- 85.255.120.194
- 211.95.72.85
- 66.232.114.139
- 72.36.162.50
- 208.72.169.133

When we run the dns command in the filter we see all the standard queries and the responses from here we can identify the domain names of the hosts above.

- 212.85.111.79 = www.homebank.pl
- 212.160.67.149 = stat.webmedia.pl
- 85.255.120.194 = winhex.org
- 211.95.72.85 = 1sense.info
- 66.232.114.139 = jezl0.com
- 72.36.162.50 =
- 208.72.169.133 =

For the last 2 lps we can see that there is no dns protocol in the filter for dns as seen in this screenshot:

Filter: ip.addr == 72.36.162.50 or ip.addr == 208.72.169.133							Expression...	Clear	Apply	Save
No.	Time	Source	Destination	Protocol	Length	Info				
447	8.57	10.0.0.130	72.36.162.50	TCP	62	c1222-acse > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1				
514	8.75	72.36.162.10.0.0.130		TCP	58	http > c1222-acse [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460				
515	8.75	10.0.0.130	72.36.162.50	TCP	54	c1222-acse > http [ACK] Seq=1 Ack=1 Win=65535 Len=0				
516	8.75	10.0.0.130	72.36.162.50	HTTP	399	GET /cgi-bin/index.cgi?support&4bec02 HTTP/1.1				
517	8.75	72.36.162.10.0.0.130		TCP	54	http > c1222-acse [ACK] Seq=1 Ack=346 Win=64240 Len=0				
562	8.95	72.36.162.10.0.0.130		TCP	1502	[TCP segment of a reassembled PDU]				
563	8.95	72.36.162.10.0.0.130		TCP	1502	[TCP segment of a reassembled PDU]				
564	8.95	10.0.0.130	72.36.162.50	TCP	54	c1222-acse > http [ACK] Seq=346 Ack=2897 Win=65535 Len=0				
565	8.95	72.36.162.10.0.0.130		TCP	1498	[TCP segment of a reassembled PDU]				
566	8.95	72.36.162.10.0.0.130		TCP	1502	[TCP segment of a reassembled PDU]				
567	8.95	10.0.0.130	72.36.162.50	TCP	54	c1222-acse > http [ACK] Seq=346 Ack=5789 Win=65535 Len=0				
570	9.13	72.36.162.10.0.0.130		TCP	1502	[TCP segment of a reassembled PDU]				
571	9.13	72.36.162.10.0.0.130		TCP	1502	[TCP segment of a reassembled PDU]				
572	9.13	72.36.162.10.0.0.130		TCP	1502	[TCP segment of a reassembled PDU]				
573	9.13	10.0.0.130	72.36.162.50	TCP	54	c1222-acse > http [ACK] Seq=346 Ack=8685 Win=65535 Len=0				
574	9.14	72.36.162.10.0.0.130		TCP	1514	[TCP segment of a reassembled PDU]				
575	9.14	72.36.162.10.0.0.130		HTTP	270	HTTP/1.1 200 OK (text/html)				
576	9.14	10.0.0.130	72.36.162.50	TCP	54	c1222-acse > http [ACK] Seq=346 Ack=11809 Win=62411 Len=0				
582	10.4	10.0.0.130	72.36.162.50	TCP	54	[TCP Window Update] c1222-acse > http [ACK] Seq=346 Ack=11809 Win=65535				
606	11.0	10.0.0.130	208.72.169.133	TCP	62	resacommunity > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1				
716	12.0	10.0.0.130	72.36.162.50	HTTP	402	GET /cgi-bin/index.cgi?f37893900100f060008004eba5067ecea2db02bb2ad8ee				
717	12.0	72.36.162.10.0.0.130		TCP	54	http > c1222-acse [ACK] Seq=11809 Ack=694 Win=64240 Len=0				

Here for HTTP content served:

We use the command:

```
!(ip.addr == 195.187.7.66 or ip.addr == 213.199.161.251 or ip.addr == 193.59.201.62) && http.content_type
```

And here is the filtered http content information.

Filter: 161.251 or ip.addr == 193.59.201.62) && http.content_t Expression... Clear Apply Save						
No.	Time	Source	Destination	Protocol	Length	Info
172	5.76	212.85.111.79	10.0.0.130	HTTP	566	HTTP/1.1 200 OK (text/html)
176	6.53	212.85.111.79	10.0.0.130	HTTP	646	HTTP/1.1 200 OK (text/css)
183	6.66	212.85.111.79	10.0.0.130	HTTP	646	[TCP Retransmission] HTTP/1.1 200 OK (text/css)
190	6.92	212.160.67.149	10.0.0.130	HTTP	1212	HTTP/1.1 200 OK (GIF87a) (GIF87a) (image/gif)
201	7.29	85.255.120.194	10.0.0.130	HTTP	596	HTTP/1.1 302 Found (text/html)
205	7.39	85.255.120.194	10.0.0.130	HTTP	596	[TCP Retransmission] HTTP/1.1 302 Found (text/html)
277	7.92	66.232.114.139	10.0.0.130	HTTP	180	HTTP/1.1 200 OK (text/html)
441	8.53	211.95.72.85	10.0.0.130	HTTP	512	HTTP/1.1 200 OK (text/html)
471	8.63	211.95.72.85	10.0.0.130	HTTP	512	[TCP Retransmission] HTTP/1.1 200 OK (text/html)
575	9.14	72.36.162.50	10.0.0.130	HTTP	270	HTTP/1.1 200 OK (text/html)
602	10.8	66.232.114.139	10.0.0.130	HTTP	305	HTTP/1.1 200 OK (application/octet-stream)
714	11.6	66.232.114.139	10.0.0.130	HTTP	714	HTTP/1.1 200 OK (application/octet-stream)
772	12.4	72.36.162.50	10.0.0.130	HTTP	154	HTTP/1.1 200 OK (text/javascript)
806	14.7	72.36.162.50	10.0.0.130	HTTP	1501	HTTP/1.1 200 OK (application/octet-stream)

From this we can assume some suspicious activity through the information we have here and here is a table of our IPs with our observations:

Source IP	Content Type	Description	Notes
212.85.111.79	text/html, text/css	Standard webpage and stylesheet.	No suspicious content observed.

Source IP	Content Type	Description	Notes
212.160.67.149	text/html, image/gif	HTML content and image files.	No suspicious binary files.
85.255.120.194	HTTP 302 found	Traffic redirection.	Possible suspicious file for further analysis.
66.232.114.139	application/octet-stream	Binary file download.	Possible suspicious file for further analysis.
211.95.72.85	text/html	Standard html content.	No suspicious content observed.
72.36.162.50	application/octet-stream	Binary file download.	Possible suspicious file for further analysis.

We see here the packets that contain application/octet-stream these are all potential binary files, often executables also http 302 found status code this shows a redirection, and this can be used in malicious traffic.

The 3 suspicious files we have noticed here are these:

- 66.232.114.139 – application/octet-stream - jezl0.com
- 72.36.162.50 – application/octet-stream – no dns
- 85.255.120.194 – 302 found - winhex.org

Using the command

```
http.content_type contains "application/octet-stream"
```

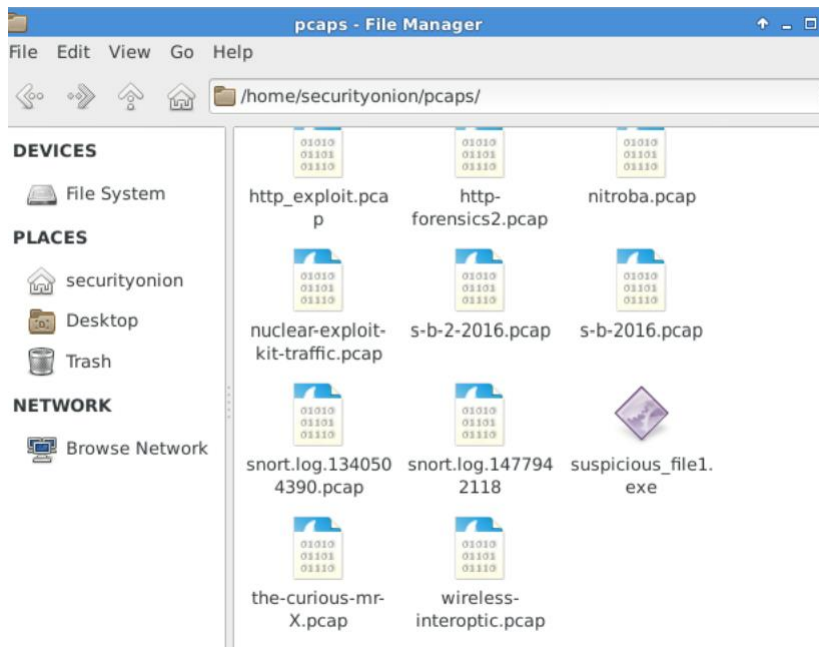
We can filter for just these tcp streams by right clicking on them and following tcp stream here we can find the last packet for both ips “66.232.114.139” and “72.36.162.50”.

Filter: http.content_type contains "application/octet-stream" Expression... Clear Apply Save							
No.	Time	Source	Destination	Protocol	Length	Info	
602	10.8s	66.232.114.139	10.0.0.130	HTTP	305	HTTP/1.1 200 OK (application/octet-stream)	
714	11.6s	66.232.114.139	10.0.0.130	HTTP	714	HTTP/1.1 200 OK (application/octet-stream)	
806	14.7s	72.36.162.50	10.0.0.130	HTTP	1501	HTTP/1.1 200 OK (application/octet-stream)	

Download #1

- Source: 72.36.162.50

- Domain Name: Not resolved.
- Content: application/octet-stream.
- Last Packet: Packet #806 (end of TCP stream).
- Exported File: suspicious_file1.exe



Download #2

- Source: 66.232.114.139
- Domain Name: jezl0.com.
- Content: application/octet-stream.
- Last Packet: Packet #602 (end of TCP stream).
- Exported File: suspicious_file2.exe



Download #3

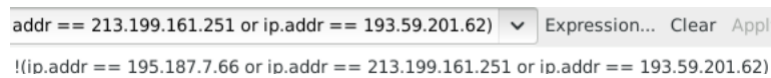
- Source: Redirect from 85.255.120.194 → Target Host (e.g., Redirected IP).
- Domain Name: Not resolved for target.
- Content: application/octet-stream.

- Last Packet: Packet #XXX (end of TCP stream for redirection target).
- Exported File: file3_redirected.exe.

Part 2 Drive by Download with FAST FLUX:

Step 1:

Filter out the benign traffic:



addr == 213.199.161.251 or ip.addr == 193.59.201.62) Expression... Clear Appl

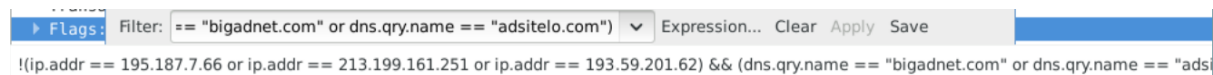
!(ip.addr == 195.187.7.66 or ip.addr == 213.199.161.251 or ip.addr == 193.59.201.62)

Step 2:

Analyze DNS for bigadnet.com and adsitelo.com:

Using the command:

```
!(ip.addr == 195.187.7.66 or ip.addr == 213.199.161.91 or ip.addr == 193.59.201.62) &&
(dns.qry.name == "bigadnet.com" or dns.qry.name == "adsitelo.com")
```



Filter: == "bigadnet.com" or dns.qry.name == "adsitelo.com") Expression... Clear Apply Save

!(ip.addr == 195.187.7.66 or ip.addr == 213.199.161.251 or ip.addr == 193.59.201.62) && (dns.qry.name == "bigadnet.com" or dns.qry.name == "adsitelo.com")

We filter for the traffic of the above websites.

Here is the DNS response packets and we see all the Ips returned here for each domain in the answer section, we can tell that it is fast-flux behaviour as multiple ip addresses are returned in the DNS responses and the Ips are often short lived and change frequently.

```

4 10.0.0.2 Standard query response 0xda3b A 91.98.94.45 A 69.66.247.232 A 80.200.239.235 A 84.10.100.196 A 122.128.253.14 A 85.2
Frame 4: 296 bytes on wire (2368 bits), 296 bytes captured (2368 bits)
Ethernet II, Src: Vmware_ed:52:57 (00:50:56:ed:52:57), Dst: Vmware_fa:18:ca (00:0c:29:fa:18:ca)
Internet Protocol Version 4, Src: 10.0.0.2 (10.0.0.2), Dst: 10.0.0.130 (10.0.0.130)
User Datagram Protocol, Src Port: domain (53), Dst Port: 1030 (1030)
Domain Name System (response)
[Request In: 3]
[Time: 0.002315000 seconds]
Transaction ID: 0xda3b
Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 14
Authority RRs: 0
Additional RRs: 0
Queries
  bigadnet.com: type A, class IN
    Name: bigadnet.com
    Type: A (Host address)
    Class: IN (0x0001)
Answers
  bigadnet.com: type A, class IN, addr 91.98.94.45
  bigadnet.com: type A, class IN, addr 69.66.247.232
  bigadnet.com: type A, class IN, addr 80.200.239.235
  bigadnet.com: type A, class IN, addr 84.10.100.196
  bigadnet.com: type A, class IN, addr 122.128.253.14
  bigadnet.com: type A, class IN, addr 85.226.168.12
  bigadnet.com: type A, class IN, addr 98.227.46.217
  bigadnet.com: type A, class IN, addr 119.30.67.167
  bigadnet.com: type A, class IN, addr 68.200.236.117
  bigadnet.com: type A, class IN, addr 85.50.79.83
  bigadnet.com: type A, class IN, addr 76.18.84.109
  bigadnet.com: type A, class IN, addr 24.91.169.202
  bigadnet.com: type A, class IN, addr 72.231.140.235
  bigadnet.com: type A, class IN, addr 75.42.216.222

```

Step 3:

Locate the Client's Connection to bigadnet.com:

Using the command (ip.addr == 91.98.94.45) we are able to locate the connection between the client and bigadnet.com

Filter: (ip.addr == 91.98.94.45) Expression... Clear Apply Save				
No.	Source	Info	Protocol	Destination
5	10.0.0.130	elfiq-repl > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	91.98.94.45
6	91.98.94.45	http > elfiq-repl [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460	TCP	10.0.0.130
7	10.0.0.130	elfiq-repl > http [ACK] Seq=1 Ack=1 Win=65535 Len=0	TCP	91.98.94.45
8	10.0.0.130	GET /cgi-bin/index.cgi?ad HTTP/1.1	HTTP	91.98.94.45
9	91.98.94.45	http > elfiq-repl [ACK] Seq=1 Ack=218 Win=64240 Len=0	TCP	10.0.0.130
169	91.98.94.45	HTTP/1.1 302 Found (text/html)	HTTP	10.0.0.130
170	10.0.0.130	elfiq-repl > http [FIN, ACK] Seq=218 Ack=415 Win=65121 Len=0	TCP	91.98.94.45
171	91.98.94.45	http > elfiq-repl [ACK] Seq=415 Ack=219 Win=64239 Len=0	TCP	10.0.0.130
172	91.98.94.45	http > elfiq-repl [FIN, PSH, ACK] Seq=415 Ack=219 Win=64239 Len=0	TCP	10.0.0.130
173	10.0.0.130	elfiq-repl > http [ACK] Seq=219 Ack=416 Win=65121 Len=0	TCP	91.98.94.45

Initial Connection:

- The client (10.0.0.130) established a connection to 91.98.94.45 over HTTP.

Requested Resource:

- The client requested: /cgi-bin/index.cgi?ad.

Response from Server:

- The server (91.98.94.45) responded with a 302 Found status code.
- Redirect Target:
 - Extract the Location header from the HTTP response packet (#169) to determine the redirection target (likely to adsitelo.com).

TCP Summary:

- Handshake:
 - Packets #5-7: TCP handshake completed successfully.
- Termination:
 - Packets #170-173: Connection cleanly closed.

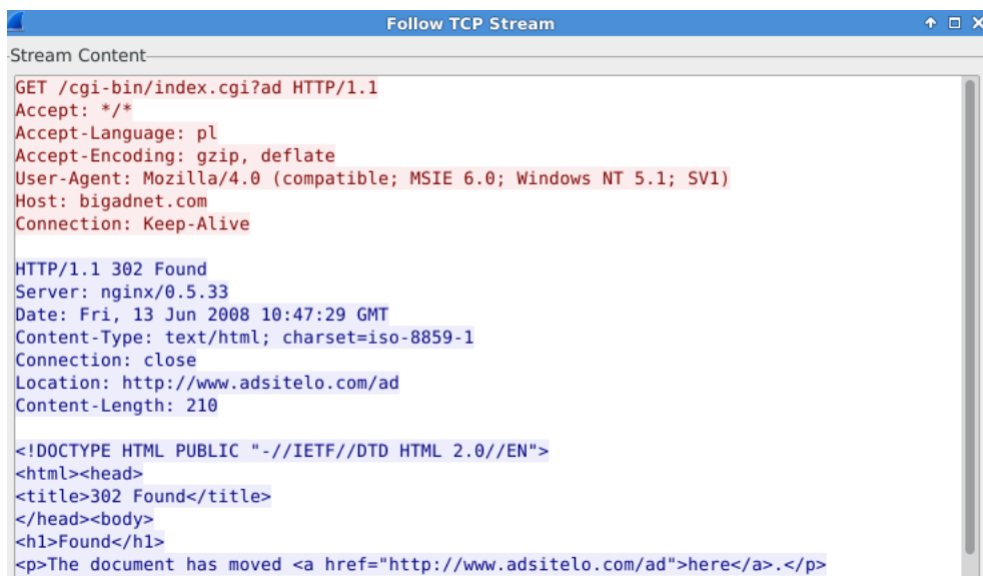
Step 4:

Identify the Redirect to adsitelo.com:

Using the command to filter for the http response code 302 “http.response.code == 302”

Filter: http.response.code == 302 Expression... Clear Apply Save					
No.	Source	Info	Protocol	Destination	
169	91.98.94.45	HTTP/1.1 302 Found (text/html)	HTTP	10.0.0.130	

Now we follow the tcp stream we right click on the packet and follow the tcp stream and here we will look at the http heder and we see the location with the url to adsitelo.com here is the screenshot of this tcp stream.



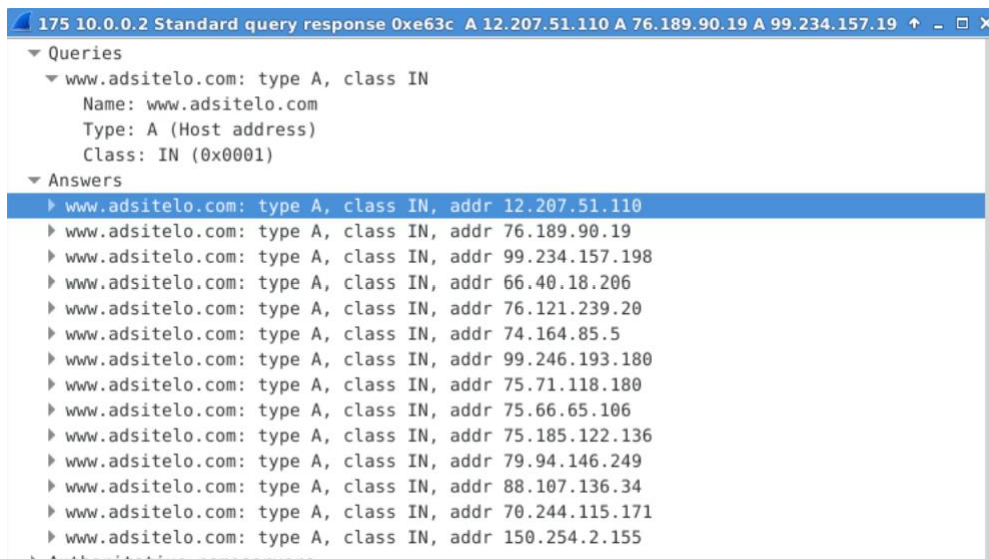
Connection Attempts to adsitelo.com:

First we search through dns for the adsitelo.com ip address and the first 2 that show and as we use the command “dns” we find the standard query and in this screenshot we see the ip addresses we will use.

174	10.0.0.130	Standard query 0xe63c A www.adsitelo.com	DNS	10.0.0.2
175	10.0.0.2	Standard query response 0xe63c A 12.207.51.110 A 76.189.90.19 A 99.234.157.19	DNS	10.0.0.130

The ip addresses we will use is “12.207.51.110” and “76.189.90.19”

Also here is the dns response for adsitelo.com with all the ips listed in here too.



Here is the first connection:

No.	Source	Info	Protocol	Destination
176	10.0.0.130	bvtsonar > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	12.207.51.110
351	10.0.0.130	[TCP Retransmission] bvtsonar > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	12.207.51.110
352	12.207.51.110	http > bvtsonar [ACK] Seq=1 Ack=1 Win=64240 Len=0	TCP	10.0.0.130
442	10.0.0.130	[TCP Retransmission] bvtsonar > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	12.207.51.110
443	12.207.51.110	[TCP Dup ACK 352#1] http > bvtsonar [ACK] Seq=1 Ack=1 Win=64240 Len=0	TCP	10.0.0.130

Here is the second connection:

No.	Source	Info	Protocol	Destination
444	10.0.0.130	blaze > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	76.189.90.19
445	10.0.0.130	[TCP Retransmission] blaze > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	76.189.90.19
446	76.189.90.19	http > blaze [ACK] Seq=1 Ack=1 Win=64240 Len=0	TCP	10.0.0.130
447	10.0.0.130	[TCP Retransmission] blaze > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PERM=1	TCP	76.189.90.19
448	76.189.90.19	[TCP Dup ACK 446#1] http > blaze [ACK] Seq=1 Ack=1 Win=64240 Len=0	TCP	10.0.0.130

As we see in there is syn packets without acknowledgments meaning the connection failed.

In the third connection we see this:

Filter: (ip.addr == 99.234.157.198) Expression... Clear Apply Save				
No.	Source	Info	Protocol	Destination
449	10.0.0.130	unizensus > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1	TCP	99.234.157.198
450	99.234.157.198	http > unizensus [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460	TCP	10.0.0.130
451	10.0.0.130	unizensus > http [ACK] Seq=1 Ack=1 Win=65535 Len=0	TCP	99.234.157.198
452	10.0.0.130	GET /ad HTTP/1.1	HTTP	99.234.157.198
453	99.234.157.198	http > unizensus [ACK] Seq=1 Ack=204 Win=64240 Len=0	TCP	10.0.0.130
454	99.234.157.198	HTTP/1.1 301 Moved Permanently (text/html)	HTTP	10.0.0.130
455	10.0.0.130	unizensus > http [ACK] Seq=204 Ack=454 Win=65083 Len=0	TCP	99.234.157.198
456	10.0.0.130	unizensus > http [FIN, ACK] Seq=204 Ack=454 Win=65083 Len=0	TCP	99.234.157.198
457	99.234.157.198	http > unizensus [ACK] Seq=454 Ack=205 Win=64239 Len=0	TCP	10.0.0.130
458	10.0.0.130	winpoplanmess > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1	TCP	99.234.157.198
459	99.234.157.198	http > winpoplanmess [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460	TCP	10.0.0.130
460	10.0.0.130	winpoplanmess > http [ACK] Seq=1 Ack=1 Win=65535 Len=0	TCP	99.234.157.198
461	10.0.0.130	GET /ad/ HTTP/1.1	HTTP	99.234.157.198
462	99.234.157.198	http > winpoplanmess [ACK] Seq=1 Ack=205 Win=64240 Len=0	TCP	10.0.0.130
463	99.234.157.198	[TCP segment of a reassembled PDU]	TCP	10.0.0.130
464	99.234.157.198	[TCP segment of a reassembled PDU]	TCP	10.0.0.130
465	10.0.0.130	winpoplanmess > http [ACK] Seq=205 Ack=1461 Win=65535 Len=0	TCP	99.234.157.198
466	99.234.157.198	HTTP/1.1 200 OK (text/html)	HTTP	10.0.0.130
467	10.0.0.130	winpoplanmess > http [ACK] Seq=205 Ack=2778 Win=64219 Len=0	TCP	99.234.157.198

This is a successful connection as we see syn packets with acknowledgements showing a successful connection. Also, towards the end of the packet we see that the malware was downloaded as we can see application/x-msdownload which is a binary file download.

564	10.0.0.130	c1222-acse > http [ACK] Seq=192 Ack=49348 Win=65535 Len=0	TCP	99.234.157.198
565	99.234.157.198	[TCP segment of a reassembled PDU]	TCP	10.0.0.130
566	99.234.157.198	[TCP segment of a reassembled PDU]	TCP	10.0.0.130
567	10.0.0.130	c1222-acse > http [ACK] Seq=192 Ack=52244 Win=65535 Len=0	TCP	99.234.157.198
568	99.234.157.198	HTTP/1.1 200 OK (application/x-msdownload)	HTTP	10.0.0.130
569	10.0.0.130	c1222-acse > http [ACK] Seq=192 Ack=52638 Win=65142 Len=0	TCP	99.234.157.198

Step 6:

Analyze the Malware Download:

Using the command http.content_type contains “application”

Filter: http.content_type contains "application" Expression... Clear Apply Save				
No.	Source	Info	Protocol	Destination
297	193.59.201.62	HTTP/1.1 200 OK (application/x-shockwave-flash)	HTTP	10.0.0.130
337	193.59.201.62	HTTP/1.1 200 OK (application/x-shockwave-flash)	HTTP	10.0.0.130
568	99.234.157.198	HTTP/1.1 200 OK (application/x-msdownload)	HTTP	10.0.0.130

Here we see that the source IP is “99.234.157.198”

Once here we right click on the application/x-msdownload and follow the tcp stream and once in here we first verify the content disposition and as we see in the screenshot below the filename is exe.exe

```
Follow TCP Stream
Stream Content
GET /ad/load.php HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1)
Host: www.adsitel.com
Connection: Keep-Alive

HTTP/1.1 200 OK
Server: nginx/0.5.33
Date: Fri, 13 Jun 2008 10:48:15 GMT
Content-Type: application/x-msdownload
Connection: close
X-Powered-By: PHP/5.2.6
Pragma: public
Expires: Fri, 13 Jun 2008 03:51:50 GMT
Cache-Control: must-revalidate, post-check=0, pre-check=0
Cache-Control: private
Content-Disposition: attachment; filename="exe.exe"
Content-Transfer-Encoding: binary
Content-Length: 52224
```

Now to check the original filename we must go to the end of this tcp stream and we will see within the binary content the original filename and we can make out here “@original filename aspimg exe”

```
.....S.t.r.i.n.g.F.i.l.e.I.n.f.o.....0.4.0.9.0.4.b.0...
L....C.o.m.p.a.n.y.N.a.m.e.....M.i.c.r.o.s.o.f.t. .C.o.r.p.o.r.a.t.i.o.n...V....F.i.l.
e.D.e.s.c.r.i.p.t.i.o.n....M.i.c.r.o.s.o.f.t. .A.S.P.I. .M.a.n.a.g.e.r....d"...F.i.l.
e.V.e.r.s.i.o.n....5...1...2.6.0.0...0. .
(x.p.c.l.i.e.n.t...0.1.0.8.1.7.-.1.1.4.8.)...0....I.n.t.e.r.n.a.l.N.a.m.e...a.s.p.i.m.
g.r.....L.e.g.a.l.C.o.p.y.r.i.g.h.t.... .M.i.c.r.o.s.o.f.t. .C.o.r.p.o.r.a.t.i.o.
n... .A.l.l. .r.i.g.h.t.s. .r.e.s.e.r.v.e.d....@....0.r.i.g.i.n.a.l.F.i.l.e.n.a.m.e...
a.s.p.i.m.g.r...e.x.e...j.
%...P.r.o.d.u.c.t.N.a.m.e.....M.i.c.r.o.s.o.f.t... .W.i.n.d.o.w.s... .0.p.e.r.a.t.i.n.
g. .S.y.s.t.e.m....h"...P.r.o.d.u.c.t.V.e.r.s.i.o.n...5...1...2.6.0.0...0. .
(x.p.c.l.i.e.n.t...0.1.0.8.1.7.-.1.1.4.8.)...D....V.a.r.F.i.l.e.I.n.f.o....
$....T.r.a.n.s.l.a.t.i.o.n.....
\4..@4.....i4..T4.....t4...4...4...4.....4.....K.E.R.N.E.L.32.D.L
L.M.S.V.C.R.T.d.l.l...L.o.a.d.L.i.b.r.a.r.y.A..G.e.t.P.r.o.c.A.d.d.r.e.s.s...V.i.r.t.u.a.l.P.r.o.t.e.c.t...E.x.i.t.P.r.o.c.e.s.s...f.r.e
e.....
```



Step 7:

For this step first we will run a dns filter, so we know the ip addresses for ns.uk2.net, yahoo.com and web.de.

Filter: dns			Expression...	Clear	Apply	Save
No.	Source	Info				
3	10.0.0.130	Standard query 0xda3b A bigadnet.com				
4	10.0.0.2	Standard query response 0xda3b A 91.98.94.45 A 69.66.247.232				
10	10.0.0.130	Standard query 0x0453 A www.cert.pl				
11	10.0.0.2	Standard query response 0x0453 CNAME melkor.nask.waw.pl A 195				
34	10.0.0.130	Standard query 0x7050 A urs.microsoft.com				
46	10.0.0.2	Standard query response 0x7050 CNAME urs.microsoft.com.nsac.				
174	10.0.0.130	Standard query 0xe63c A www.adsitelo.com				
175	10.0.0.2	Standard query response 0xe63c A 12.207.51.110 A 76.189.90.19				
177	10.0.0.130	Standard query 0x1156 A www.nask.pl				
178	10.0.0.2	Standard query response 0x1156 A 193.59.201.62				
572	10.0.0.130	Standard query 0xbe3c A ns.uk2.net				
573	10.0.0.2	Standard query response 0xbe3c CNAME uk2.net A 83.170.69.14				
575	10.0.0.130	Standard query 0x013d A www.yahoo.com				
576	10.0.0.2	Standard query response 0x013d CNAME www.yahoo-ht3.akadns.net				
581	10.0.0.130	Standard query 0x8e3e A www.web.de				
582	10.0.0.2	Standard query response 0x8e3e A 217.72.195.42				

Then we will filter specifically for each one now:

Ns.uk2.net:

Filter: (ip.addr == 83.170.69.14)			Expression...	Clear	Apply	Save
Info	Protocol	Destination				
resacomcommunity > domain [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK TCP		83.170.69.14				

Filter: (ip.addr == 83.170.69.14)		
No.	Source	
574	10.0.0.130	

**had to take 2 screenshots line was too long

From this file we learned that no ack was received from destination, sent by packet 574.

Yahoo.com:

For yahoo we had to open up the response to get a hold of the IP which we found in the answer section of the packet.

576 10.0.0.2 Standard query response 0x013d CNAME www.yahoo-ht3.akadns.net A 87.248.11			↑	▢	×
[Time: 0.191272000 seconds]					
Transaction ID: 0x013d					
Flags: 0x8180 Standard query response, No error					
Questions: 1					
Answer RRs: 2					
Authority RRs: 9					
Additional RRs: 9					
Queries					
Answers					
www.yahoo.com: type CNAME, class IN, cname www.yahoo-ht3.akadns.net					
Name: www.yahoo.com					
Type: CNAME (Canonical name for an alias)					
Class: IN (0x0001)					
Time to live: 5 minutes					
Data length: 26					
Primaryname: www.yahoo-ht3.akadns.net					
www.yahoo-ht3.akadns.net: type A, class IN, addr 87.248.113.14					
Name: www.yahoo-ht3.akadns.net					

The ip being “87.248.113.14”

Once we type in the filter (ip.addr == 87.248.113.14) we find the packets.

Filter:		(ip.addr == 87.248.113.14)	▼	Expression...	Clear	Apply	Save
No.	Source	Info					
577	10.0.0.130	nfa > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1					
578	87.248.113.14	http > nfa [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460					
579	10.0.0.130	nfa > http [ACK] Seq=1 Ack=1 Win=65535 Len=0					
580	10.0.0.130	nfa > http [RST, ACK] Seq=1 Ack=1 Win=0 Len=0					

Now we will locate the packet the client reset the connection with. To find this we will add onto the filter we used the tcp flag reset and = it to 1 “(ip.addr == 87.248.113.14) && (tcp.flags.reset ==1)” and we get the packet that the client used to reset the connection.

Filter:	(ip.addr == 87.248.113.14) && (tcp.flags.reset ==1)	▼	Expression...	Clear	Apply	Save
No.	Source	Info				
580	10.0.0.130	nfa > http [RST, ACK] Seq=1 Ack=1 Win=0 Len=0				

▶ Frame 580: 54 bytes on wire (432 bits), 54 bytes captured (432 bits)

▶ Ethernet II, Src: Vmware_fa:18:ca (00:0c:29:fa:18:ca), Dst: Vmware_ed:52:57 (00:50:56:ed:52:57)

▶ Internet Protocol Version 4, Src: 10.0.0.130 (10.0.0.130), Dst: 87.248.113.14 (87.248.113.14)

▼ Transmission Control Protocol, Src Port: nfa (1155), Dst Port: http (80), Seq: 1, Ack: 1, Len: 0

Source port: nfa (1155)

Destination port: http (80)

[Stream index: 13]

Sequence number: 1 (relative sequence number)

Acknowledgment number: 1 (relative ack number)

Header length: 20 bytes

▼ Flags: 0x014 (RST, ACK)

000. = Reserved: Not set

...0 = Nonce: Not set

.... 0... = Congestion Window Reduced (CWR): Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...1 = Acknowledgment: Set

.... 0... = Push: Not set

▶ 1.. = Reset: Set

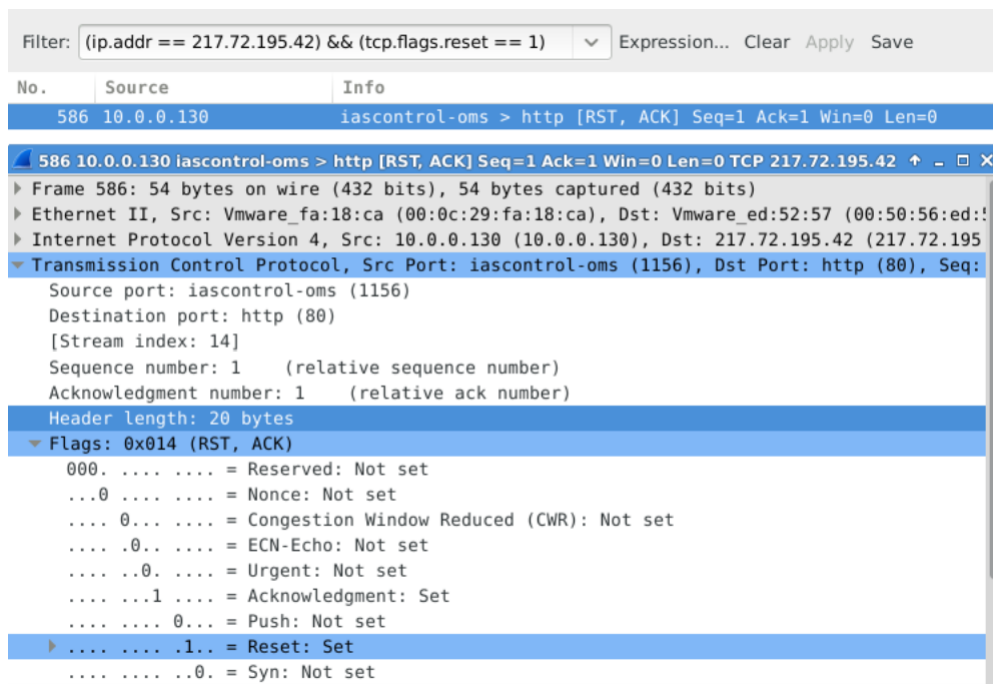
Here we verify that this packet had reset the connection.

Web.de:

The ip for web.de is 217.72.195.42 as we know this from the previous dns filter, 217.72.195.42

Filter:	(ip.addr == 217.72.195.42)	▼	Expression...	Clear	Apply	Save
No.	Source	Info				
583	10.0.0.130	iascontrol-oms > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1				
584	217.72.195.42	http > iascontrol-oms [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460				
585	10.0.0.130	iascontrol-oms > http [ACK] Seq=1 Ack=1 Win=65535 Len=0				
586	10.0.0.130	iascontrol-oms > http [RST, ACK] Seq=1 Ack=1 Win=0 Len=0				

Now to verify the reset packet by the client we do the same like last and add to the filter we already have “&& (tcp.flags.reset == 1)” and now we see the exact reset packet that was used by the client.



As we see in this screenshot above from the filter, we use we identify the reset packet which the packet number is 586 and when we open the packet we can see in the flags section that the reset is set as it has a value of 1.

216.150.79.226:

1. Connection Establishment

- TCP Handshake:
 - Packet #587: Client (10.0.0.130) sends SYN to server (216.150.79.226).
 - Packet #588: Server (216.150.79.226) responds with SYN-ACK.
 - Packet #589: Client (10.0.0.130) sends ACK, completing the handshake.

Filter: (ip.addr == 216.150.79.226)		▼	Expression... Clear Apply Save		
No.	Source	Info	Protocol	Destination	
587	10.0.0.130	iascontrol > http [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK PER	TCP	216.150.79.226	
588	216.150.79.226	http > iascontrol [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=14	TCP	10.0.0.130	
589	10.0.0.130	iascontrol > http [ACK] Seq=1 Ack=1 Win=65535 Len=0	TCP	216.150.79.226	
590	10.0.0.130	[TCP segment of a reassembled PDU]	TCP	216.150.79.226	
591	216.150.79.226	http > iascontrol [ACK] Seq=1 Ack=314 Win=64240 Len=0	TCP	10.0.0.130	
592	10.0.0.130	[TCP segment of a reassembled PDU]	TCP	216.150.79.226	
593	216.150.79.226	http > iascontrol [ACK] Seq=1 Ack=385 Win=64240 Len=0	TCP	10.0.0.130	
594	10.0.0.130	[TCP segment of a reassembled PDU]	TCP	216.150.79.226	
595	216.150.79.226	http > iascontrol [ACK] Seq=1 Ack=401 Win=64240 Len=0	TCP	10.0.0.130	
596	10.0.0.130	POST /forum.php HTTP/1.1 (application/octet-stream)	HTTP	216.150.79.226	
597	216.150.79.226	http > iascontrol [ACK] Seq=1 Ack=1297 Win=64240 Len=0	TCP	10.0.0.130	
598	216.150.79.226	HTTP/1.1 200 OK	HTTP	10.0.0.130	
599	10.0.0.130	iascontrol > http [ACK] Seq=1297 Ack=1076 Win=64461 Len=0	TCP	216.150.79.226	
600	10.0.0.130	iascontrol > http [FIN, ACK] Seq=1297 Ack=1076 Win=64461 Len=0	TCP	216.150.79.226	
601	216.150.79.226	http > iascontrol [ACK] Seq=1076 Ack=1298 Win=64239 Len=0	TCP	10.0.0.130	

We can also verify off this that there was a data transfer on packet 596 as it sends a post to /forum.php which is an application/octet-stream. And packet 598 confirms a successful reception and a response to it. Finally, from 599 to 601 we see the connection timeout as we see the fin ack and an ack in return to that request.

Step 8:

1. First we will filter traffic for forum.php and to do that we used the filter command "http.request.uri contains "forum.php" " after that we inspected the packet and see what data the client sent to through the php script to verify this we go to the very end of the packet and see the content-dispositions filename is called DEBUG.txt and the content type is application/octet-stream and I included with the screenshot the raw binary packet bytes pane of the DEBUG.txt.

Content-Disposition: form-data; name="DEBUG"; filename="DEBUG.TXT"\r\n	
Content-Type: application/octet-stream\r\n\r\n	
Media Type	
Media Type: application/octet-stream (100 bytes)	
Last boundary: \r\n--1BEF0A57BE110FD467A--\r\n	
0420	41 0d 0a 43 6f 6e 74 65 6e 74 2d 44 69 73 70 6f A..Conte nt-Dispo
0430	73 69 74 69 6f 6e 3a 20 66 6f 72 6d 2d 64 61 74 sition: form-dat
0440	61 3b 20 6e 61 6d 65 3d 22 44 45 42 55 47 22 3b a; name= "DEBUG";
0450	20 66 69 6c 65 6e 61 6d 65 3d 22 44 45 42 55 47 filename="DEBUG
0460	2e 54 58 54 22 0d 0a 43 6f 6e 74 65 6e 74 2d 54 .TXT"..C ontent-T
0470	79 70 65 3a 20 61 70 70 6c 69 63 61 74 69 6f 6e ype: app lication
0480	2f 6f 63 74 65 74 2d 73 74 72 65 61 6d 0d 0a 0d /octet-s tream...
0490	0a 60 00 00 00 11 1b 1b 1b 19 1b 1b 1b 03 1b 1b
04a0	1b 11 1b 1b 1b 1b 1b 1b 1b 1b 1b 1b 1b 1b 1b
04b0	1b 19 1b 1b 1b 1b 03 1b 1b 1b 1b 1b 1b 1b 1b

2. Next we will locate the servers response to the http request and we find that in our old filter "(ip.addr == 216.150.79.226)" and once we inspect the packet which should have HTTP/1.1 200 OK in the info part we will navigate to the bottom of this and verify the content disposition which is common.bin


```
598 216.150.79.226 HTTP/1.1 200 OK HTTP 10.0.0.130 63.031468 1128
HTTP/1.1 200 OK\r\n
Server: nginx/0.5.33\r\n
Date: Fri, 13 Jun 2008 03:53:44 GMT\r\n
Content-Type: multipart/form-data; boundary="1BEF0A57BE110FD467A"\r\n
Connection: close\r\n
Content-Length: 889\r\n
\r\n
[HTTP response 1/1]
[Time since request: 0.393085000 seconds]
[Request in frame: 596]
MIME Multipart Media Encapsulation, Type: multipart/form-data, Boundary: "1BEF0A57BE110FD467A"
[Type: multipart/form-data]
First boundary: --1BEF0A57BE110FD467A\r\n
Encapsulated multipart part: (application/octet-stream)
Content-Disposition: form-data; name="COMMON"; filename="COMMON.BIN"\r\n
Content-Type: application/octet-stream\r\n\r\n\r\n
```

We also will verify the presence of the common.bin in the binary data pane underneath.

```
598 216.150.79.226 HTTP/1.1 200 OK HTTP 10.0.0.130 63.031468 1128
[HTTP response 1/1]
[Time since request: 0.393085000 seconds]
[Request in frame: 596]
MIME Multipart Media Encapsulation, Type: multipart/form-data, Boundary: "1BEF0A57BE110FD467A"
[Type: multipart/form-data]
First boundary: --1BEF0A57BE110FD467A\r\n
Encapsulated multipart part: (application/octet-stream)
Content-Disposition: form-data; name="COMMON"; filename="COMMON.BIN"\r\n
Content-Type: application/octet-stream\r\n\r\n\r\n
0100 34 36 37 41 0d 0a 43 6f 6e 74 65 6e 74 2d 44 69 46 7a 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
0110 73 70 6f 73 69 74 69 6f 6e 3a 20 66 6f 72 6d 2d 69 6f 6e 3a 20 66 6f 72 6d 2d 69 6f 6e 3a 20 66 6f 72 6d 2d
0120 64 61 74 61 3b 20 6e 61 6d 65 3d 22 43 4f 4d 4d 64 61 74 61 3b 20 6e 61 6d 65 3d 22 43 4f 4d 4d 64 61 74 61 3b 20 6e 61
0130 4f 4e 22 3b 20 66 69 6c 65 6e 61 6d 65 3d 22 43 4f 4d 4d 64 61 74 61 3b 20 6e 61 6d 65 3d 22 43 4f 4d 4d 64 61 74 61 3b 20 6e 61
0140 4f 4d 4d 4f 4e 2e 42 49 4e 22 0d 0a 43 6f 6e 74 65 6e 74 2d 44 69 46 7a 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
0150 65 6e 74 2d 54 70 70 65 3a 20 61 70 70 6f 60 62 65 6e 74 2d 54 70 70 65 3a 20 61 70 70 6f 60 62 65 6e 74 2d 54 70 70 65 3a 20 61 70 70 6f 60 62
```

3. Finally for this step we will follow the tcp stream:

So first off we will right click on our packet 596 and follow tcp stream and once in here we scroll down and we notice the client to server which is the debug.txt and right after that then we see our response from the server to the client including the binary data from the COMMON.BIN. so overall what happens in this tcp stream is client sends debug.txt to fourm.php and server responds to client with COMMON.BIN.

```

Stream Content
5#2#2#0#2600#0
--1BEF0A57BE110FD467A
Content-Disposition: form-data; name="DEBUG"; filename="DEBUG.TXT"
Content-Type: application/octet-stream

.....
--1BEF0A57BE110FD467A--
HTTP/1.1 200 OK
Server: nginx/0.5.33
Date: Fri, 13 Jun 2008 03:53:44 GMT
Content-Type: multipart/form-data; boundary="--1BEF0A57BE110FD467A"
Connection: close
Content-Length: 889

--1BEF0A57BE110FD467A
Content-Disposition: form-data; name="COMMON"; filename="COMMON.BIN"
Content-Type: application/octet-stream

'hr.%,./"(*+~*//)/', '4hr.%. 'ywtxp%. 'm%. /) - . '4m%. 'h%)+(5*,5*,+5/.) *-5*.
+5, "5))-./5**5*/5#..-/5**5("5#..-5*",5)((5*( (.-5**"5)/5"#..-5)(5*+)5*- " '4h%. ' .Dz
%,)5)(5*/+5)(..)/5**5*- "5)+.##5)(+5"#5...*)5*)#5). (5*/. "5"#5"/5/..#/5*+5*+5*- .#.5.
+5, "5#(.-"5--5)/,5)(. #.5))-5*-#5*)..#5)++5)(-5**,. "#5),5/-5)*,..5/5)*-5)))..**5

```

Entire conversation (2370 bytes)

Find Save As Print ASCII EBCDIC Hex Dump C Arrays Raw

QUESTION:

why would blackholing the IP address(es) of bigadnet.com not be effective here? What would you suggest as an alternative to blackholing the IP address(es)?

Why this would not be effective is for a few reasons:

- Redirection behaviour when we saw the HTTP 302 redirection bigadnet.com redirects the client to a different domain, so even if the Ips are to be blocked the client can still be just redirected to different malicious servers.
- Fast Flux DNS is also another big reason as to why it would be ineffective as we know it was in use with the multiple IP addresses in the response for the DNS and the domain resolves to a set of frequently changing pool of addresses.so backholing them would be ineffective as new ips will replace the back held ones.

Solutions:

- Simply use Domain based blocking so instead of just blocking ip addresses you can block the domain so in this case bigadnet.com at the dns level.
- Another idea would be to monitor and then block redirections so by monitoring http traffic for redirection attempts then block access to domains for example in what we did an example would be to block adsitelo.com.

Part 3 Tshark:

```
securityonion@so:~/pcaps$ tshark -n -r dbdt.pcap -R 'ip' -T fields -e 'ip.src' -  
e 'ip.dst' | tr '\t' '\n' | sort | uniq  
tshark: -R without -2 is deprecated. For single-pass filtering use -Y.  
10.0.0.130  
10.0.0.2  
172.16.241.255  
193.59.201.62  
195.187.7.66  
208.72.169.133  
211.95.72.85  
212.160.67.149  
212.85.111.79  
213.199.161.251  
66.232.114.139  
72.36.162.50  
85.255.120.194  
securityonion@so:~/pcaps$ tshark -n -r dbdtff.pcap -R 'ip' -T fields -e 'ip.src'  
-e 'ip.dst' | tr '\t' '\n' | sort | uniq  
tshark: -R without -2 is deprecated. For single-pass filtering use -Y.  
10.0.0.130  
10.0.0.2  
12.207.51.110  
193.59.201.62  
195.187.7.66  
213.199.161.251  
216.150.79.226  
217.72.195.42  
76.189.90.19  
83.170.69.14  
87.248.113.14  
91.98.94.45  
99.234.157.198  
securityonion@so:~/pcaps$
```

Using tshark we created a list of all the unique IP addresses involved within both pcap files dbdt.pcap and dbdtff.pcap and in the screenshot above we can see the lists of IP addresses we used the following command to filter this and we used '|' pipe to filter so we didn't have any duplicates and have them ordered in order and also have them formatted 1 per line rather than 2 per line.

Command:

First I moved into the pcaps directory:

"cd pcaps"

Then I used the command to get my output from the screenshot above:

For dbdt.pcap:

"tshark -n -r dbdt.pcap -T fields -e ip.src -e ip.dst | tr '\t' '\n' | sort | uniq"

For dbdtff.pcap:

"tshark -n -r dbdtff.pcap -T fields -e ip.src -e ip.dst | tr '\t' '\n' | sort | uniq"

